

令和〇年度 修士論文

マルチエージェント議論における  
チームレベル適応度を用いた  
LoRA エージェント集団の進化的最適化

Evolutionary Optimization of LoRA Agent Populations  
with Team-Level Fitness in Multi-Agent Debate

2026 年 9 月 30 日

指導教員 余俊 助教

新潟大学 自然科学研究科  
電気情報工学専攻 情報社会デザイン科学コース

学生番号 F25C142E

舩田 岳

Gaku Masuda



# 概要

大規模言語モデル (LLM) の複数体を議論させるマルチエージェント推論は、単一モデルの限界を超える手段として注目されるが、小型モデルでは「計算量を揃えた多数決 (Self-Consistency) に勝てない」という強い批判がある。本研究は、4B 級モデル (Qwen3-4B-Instruct-2507) に LoRA で異なるペルソナを焼き込んだ 3 体のエージェント集団を構成し、(1) 議論によるチーム推論が生成回数を揃えた Self-Consistency@9 (SC@9) を上回れるか、(2) チームレベル適応度 (厳密 Shapley 値  $\times$  fitness sharing) による重み空間の進化的最適化が寄与するかを、3 ベンチマーク (MMLU-Pro  $\cdot$  MATH-500  $\cdot$  SuperGPQA)  $\times$  3 シードの対応あり検定 (問題 ID クラスタリングを考慮した置換検定  $\cdot$  ブートストラップ + Holm 補正) で検証した。

実験 1 (主実験) の結果、議論の効果はベンチマーク依存に反転することが判明した。素の議論は数学 (+4.2pt) と高難度科学 (+3.1pt) で有意に有効だが MMLU-Pro では有意に有害 (-4.1pt) であり、重みペルソナチームは逆に MMLU-Pro のみで有効 (対ベース +4.5pt) だった。この反転は「多様性の利得」と「ペルソナ注入による能力毀損」の 2 軸トレードオフとして整理できる (ただし反転を問題の検証可能性に帰する解釈はベンチマーク内のカテゴリ別分析では支持されず、仮説にとどまる)。診断分析により、(i) ペルソナ SFT が思考連鎖を半減させ長い導出能力を選択的に破壊すること、(ii) 3 体の誰かが正解している問題の最大 13% 分を多数決集約が取りこぼすこと、(iii) 進化は適応度測定ノイズ (100 問で  $\pm 4.9$ pt) 下の勝者の呪いにより選抜として機能しなかったうえ、進化前後の  $\Delta W$  が cosine 類似度 0.9997 以上とほぼ同一——すなわち進化演算子が重み空間をほとんど動かしていなかったことを明らかにした。

実験 2 では診断に基づく処方——ベースモデル自己生成の検証済み長 CoT リプレイ混合 (38%) による再学習、自己誤り特定時のみ回答変更を許す条件付き議論、発言匿名化、logprob 重み付き投票——を、事前定義した合格基準を持つ実測ゲート方式で検証した。この過程で評価コンテナイメージ間の系統差 (同一問題  $\cdot$  同一設定で +6pt 級、ベースモデルの素の生成に選択的に作用) を検出したため、主要 4 条件を同一環境で全面再測定して統制した。最終評価において、処方はペルソナ SFT が生んだ能力毀損をベース単体との同等性 (等価マージン  $\pm 2$ pt) が成立する水準まで回復させ (対旧チーム +2.7pt,  $p = 0.0003$ ; MATH-500 では対ベース +3.4pt 有意優位)、一方で SC@9 には -3.5pt ( $p < 10^{-4}$ ) と有意に及ばなかった。

本研究の貢献は、(a) 小型 LLM 議論チームの成否を 2 軸トレードオフと損失分解で説明する診断枠組み、(b) 進化ループの機能不全（探索・多様性圧・選抜の三重の不全）の実測による特定と選抜ノイズの定量化、(c) 能力毀損を回復させる処方の実証と、それを尽くしてもなお生成回数を揃えた多数サンプリングに及ばないという統制された負の結果、(d) 評価環境の系統差がベースライン側を選択的に過小評価しチーム手法を見かけ上優位にするバイアスの発見、である。数学（MATH-500）では議論チームがベース単体を有意に上回り続けており、マルチエージェント議論の価値が残る条件を実測に基づいて画定した。



# 目次

|                       |           |
|-----------------------|-----------|
| <b>第1章 序論</b>         | <b>10</b> |
| 1.1 研究背景              | 10        |
| 1.1.1 小型言語モデルの集団による推論 | 10        |
| 1.1.2 マルチエージェント議論への批判 | 11        |
| 1.1.3 ペルソナの設計の恣意性     | 11        |
| 1.2 予備研究と本研究の出発点      | 11        |
| 1.3 研究課題              | 13        |
| 1.4 提案手法の概要           | 13        |
| 1.5 本研究の貢献            | 14        |
| 1.6 本論文の構成            | 15        |
| <b>第2章 関連研究</b>       | <b>16</b> |
| 2.1 マルチエージェント議論と集約    | 16        |
| 2.1.1 議論の原典と批判        | 16        |
| 2.1.2 同調と多様性          | 17        |
| 2.1.3 回答の集約           | 18        |
| 2.2 ペルソナと役割           | 18        |
| 2.2.1 ペルソナ付与の効果と限界    | 18        |
| 2.2.2 人間のチーム役割理論      | 19        |
| 2.2.3 役割の割り当てと選抜      | 20        |
| 2.3 議論と自己生成データからの学習   | 20        |
| 2.4 進化的モデル最適化         | 22        |
| 2.4.1 重み空間の集団最適化      | 22        |
| 2.4.2 重み合成の前提条件       | 23        |
| 2.5 チームへの貢献の定量化       | 24        |
| 2.6 評価の信頼性            | 25        |
| 2.7 本研究の位置づけ          | 26        |

|            |                          |           |
|------------|--------------------------|-----------|
| <b>第3章</b> | <b>提案手法</b>              | <b>29</b> |
| 3.1        | 問題設定と記号                  | 29        |
| 3.1.1      | ベースモデルと LoRA 個体          | 29        |
| 3.1.2      | 役割・エージェント・社会             | 31        |
| 3.1.3      | 連合と特性関数                  | 31        |
| 3.2        | 世代 0: チーム役割理論に基づくペルソナの選定 | 31        |
| 3.2.1      | 候補の作り方                   | 31        |
| 3.2.2      | 選定の手続き                   | 34        |
| 3.3        | 議論のプロトコル                 | 36        |
| 3.3.1      | ラウンドの構造                  | 36        |
| 3.3.2      | 回答の切り詰め                  | 38        |
| 3.3.3      | 議論のゲーティング                | 38        |
| 3.3.4      | 集約                       | 38        |
| 3.4        | チーム貢献度                   | 39        |
| 3.4.1      | 代表チーム文脈における厳密 Shapley 値  | 39        |
| 3.4.2      | Shapley 値が統合するもの         | 40        |
| 3.4.3      | Banzhaf 値                | 40        |
| 3.5        | 進化的社会学習                  | 41        |
| 3.5.1      | 社会の議論                    | 41        |
| 3.5.2      | 学習データ                    | 41        |
| 3.5.3      | LoRA の学習                 | 42        |
| 3.5.4      | 兄弟交叉                     | 42        |
| 3.5.5      | 選抜                       | 43        |
| 3.5.6      | アルゴリズム                   | 43        |
| 3.6        | 計算量の分析                   | 43        |
| <b>第4章</b> | <b>実験設定</b>              | <b>46</b> |
| 4.1        | モデルと実行環境                 | 46        |
| 4.1.1      | 推論                       | 46        |
| 4.1.2      | 学習                       | 47        |
| 4.1.3      | 実行環境の固定と非決定性             | 47        |
| 4.2        | ベンチマークと問題集合              | 47        |
| 4.2.1      | ベンチマーク                   | 47        |

|            |                  |           |
|------------|------------------|-----------|
| 4.2.2      | 問題集合             | 48        |
| 4.3        | 採点               | 49        |
| 4.3.1      | 回答の抽出と判定         | 49        |
| 4.3.2      | 予備研究の採点器の監査      | 50        |
| 4.4        | 系統と比較条件          | 51        |
| 4.4.1      | 進化の系統            | 51        |
| 4.4.2      | 単一モデルの自己学習 (RFT) | 52        |
| 4.4.3      | 基準               | 52        |
| 4.4.4      | 生成の回数と出力トークン数の考慮 | 53        |
| 4.5        | 評価指標と統計          | 53        |
| 4.5.1      | 評価指標             | 53        |
| 4.5.2      | 主要な比較と検定         | 53        |
| 4.5.3      | 副次的な分析           | 54        |
| 4.6        | ハイパーパラメータと設計値    | 55        |
| 4.6.1      | ハイパーパラメータ        | 55        |
| 4.6.2      | パイロットで決めた設計値     | 55        |
| 4.6.3      | 事前登録からの変更        | 57        |
| 4.7        | 計算資源と予算          | 58        |
| 4.7.1      | 計算資源             | 58        |
| 4.7.2      | 予算と中止規則          | 59        |
| <b>第5章</b> | <b>実験結果</b>      | <b>60</b> |
| 5.1        | 予備研究の再検証         | 60        |
| 5.1.1      | 予備研究の構成          | 60        |
| 5.1.2      | 修正した採点器による再採点    | 60        |
| 5.1.3      | 予備研究が失敗した理由      | 61        |
| 5.1.4      | 本研究への示唆          | 62        |
| 5.2        | 世代0のペルソナの選定      | 63        |
| 5.3        | 進化的社会学習          | 63        |
| 5.4        | 基準との比較           | 63        |
| 5.5        | 議論と選抜の分析         | 63        |

|                              |    |
|------------------------------|----|
| 第 6 章 考察                     | 64 |
| 6.1 本研究の限界 . . . . .         | 64 |
| 第 7 章 結論                     | 66 |
| 謝辞                           | 67 |
| 参考文献                         | 68 |
| 付 録 A 実験で用いたプロンプト            | 78 |
| A.1 システムプロンプトと回答書式 . . . . . | 78 |
| A.2 ペルソナ文 . . . . .          | 78 |
| A.3 round1 のプロンプト . . . . .  | 80 |

# 第1章 序論

## 1.1 研究背景

### 1.1.1 小型言語モデルの集団による推論

大規模言語モデル（LLM）の性能は、パラメータ数、学習データ、計算量の拡大とともに向上してきた。しかし規模の拡大は学習と推論の費用を急速に押し上げるため、一つのモデルを大きくし続ける方法には経済的な限界がある。そこで、複数のモデル（または同じモデルの複数のインスタンス）を相互作用させ、集団として単体を超える性能を引き出す方向が注目されている。複数の回答を生成して多数決をとるだけでも、エージェントの数に応じて精度が伸びる [1]。複数の LLM が互いの回答を読んで自分の回答を更新するマルチエージェント議論（Multi-Agent Debate; MAD）[2, 3] は、その代表的な方法である。

集団による推論は、小型のモデルにとって特に意味が大きい。約 40 億パラメータ（以下、4B 級）のモデルは、単体では最先端の大規模モデルに及ばないが、1 枚の GPU で多数のインスタンスを同時に動かせる。さらに LoRA[4] を用いれば、一つのベースモデルの上に軽量なアダプタを載せ替えるだけで、性格や役割の異なるエージェントを多数作れる。本研究は、Qwen3-4B-Instruct-2507[5] を共有のベースモデルとし、役割ごとの LoRA を持つエージェントの集団を対象とする。

本研究の着想は人間の社会にある。人間の集団では、異なる視点や性格を持つ人が議論を通じて誤りを正し合う。個人は自分の経験から学ぶだけでなく、他者の成功を観察してそれを取り入れる（社会学習）。集団は、そのように蓄積した知識を世代を越えて受け渡し、時間とともに高度になる（文化的進化 [6]）。本研究は、この「対話→学習→世代交代」の循環を LLM のエージェントの社会で実装し、それによって社会の問題解決の精度が上がるかを検証する。

### 1.1.2 マルチエージェント議論への批判

MAD の効果には強い留保が付いている．計算量を揃えた比較では，MAD は複数の推論経路の多数決をとる Self-Consistency (SC) [7] に安定して勝てない [8, 9]．議論による信念の更新は期待値の上で正答率を改善せず，利得の大半は最後の多数決で説明できるという分析もある [10]．小型のモデルでは状況はさらに厳しく，1.5B～7B のモデルと数学の課題による比較では，同じトークン数の反復サンプリングより確実に良い手法は一つもなかった [11]．同じモデルの複製同士の議論では，他者への同調が支配的になり，チームの誰かが正解していても最終回答が誤ることが多い [12]．

これらの批判から，本研究は二つの要請を引き出す．第一に，集団による精度の向上を主張するには，生成回数と出力トークン数を考慮した SC と比べなければならない．第二に，エージェントを学習させる手法の効果を主張するには，同じデータで単一のモデルを自己学習させた場合と比べなければならない．エージェントが学習して強くなったのか，単に同じ問題で学習したから強くなったのかを区別するためである．

### 1.1.3 ペルソナの設計の恣意性

MAD では，エージェントに「批判的な検証者」「創造的な発想家」のような異なるペルソナ（人物設定）を与えることが多い．しかし，性格を表すラベルだけのペルソナは，客観的な正解のある課題の精度をほとんど変えない [13, 14]．一方，エージェントごとに異なる推論の手続きを割り当てると，同じ手続きのエージェント同士の議論より精度が上がるという報告がある [15]．どのペルソナを何体組み合わせるかは，チームの成績を左右しうる設計変数であるにもかかわらず，多くの研究では直観で決められている．著者の予備研究（1.2 節）も，3 体のペルソナを直観で決めていた．

## 1.2 予備研究と本研究の出発点

著者は 2026 年 7 月に予備研究を行った．予備研究では，性格を表す日本語のペルソナ文を持つ 3 体の LoRA（批判的検証者，実務的意思決定者，発散的探索者）を対話データで学習させ，議論のチームを作った．さらに，チームへの貢献（厳

密 Shapley 値) を適応度として, LoRA の重みを交叉と突然変異で進化させた. その結果, チームは生成回数を揃えた SC@9 に有意に及ばず, 進化による改善も確認できなかった.

本研究の準備として, 予備研究のコードと保存済みのすべての生出力を再検証した (第 5 章). その結果, 予備研究は「チームへの貢献による進化」を検証できる状態になかったことが分かった. 理由は次の 4 点である.

1. **ペルソナの学習が能力を損ない, 多様性を生まなかった:** 短い応答からなる対話データでの学習が推論の記述を約半分に縮め, 長い導出を要する数学の問題の精度を下げていた. また, 異なるペルソナ間の回答の不一致率は, 同じエージェントを乱数だけ変えて再サンプリングした場合と区別できなかった.
2. **議論が同調に支配されていた:** 議論の実装がエージェントに自分の前回の回答を提示していなかった. そのため, round0 で少数派だったエージェントは, 自分が正解でも 81~95% の確率で誤った多数派に答えを変えていた.
3. **進化が選抜として機能していなかった:** 初期の集団には遺伝的な多様性がなく, 交叉と突然変異は重みをほとんど動かしていなかった (進化の前後で  $\Delta W$  のコサイン類似度 0.9996 以上). さらに, 100 問で測る適応度の誤差が候補間の差を上回っており, 選抜はノイズの判定になっていた.
4. **採点器に条件によって偏った欠陥があった:** 回答の抽出の誤りにより, MMLU-Pro でベースモデルの回答の約 1 割を誤って「A」と読んでいた. また, 数学の回答の同値判定が不十分で, 予測の約 6.5% で正しい回答を誤答と判定していた. 前者の誤りはベースモデル系の条件にだけ強く現れ, 条件間の比較を歪めていた.

これらの問題は, それぞれ本研究の設計原理になった. ペルソナは性格のラベルではなく推論の手続きとして定義し, チームとしての成績で選ぶ. 議論では自分の前回の回答を提示する. 進化では, 個体が議論の中で正解に至った発話から学ぶことで, 候補間に意味のある差を作る. 選抜には十分な数の問題を用い, 最終評価の問題とは分ける. 採点器は監査して評価の前に固定し, 統計の計画は結果を見る前に固定する.

## 1.3 研究課題

本研究は次の4つの研究課題に答える。

**RQ1（進化の効果）** 進化的社会学習を世代を重ねて行くと，社会（3体のペルソナの議論チーム）の未見問題での精度は，世代0の社会より上がるか。

**RQ2（選抜の効果）** チームへの貢献（厳密 Shapley 値）による選抜は，選抜を行わない社会学習より高い精度をもたらすか。

**RQ3（適応度の水準）** チームへの貢献による選抜は，個体の単独の精度による選抜より優れるか。

**RQ4（位置づけ）** 最終世代の社会は，ベースモデル単体，生成回数を考慮した SC，同じ問題で単一のモデルを自己学習させた場合と比べて，精度と計算量の上でどこに位置するか。

RQ1～RQ3 に対応する3つの主要な比較（H1～H3）と検定の方法，問題集合，中止規則は，本実験の結果を見る前に固定した（第4章）。

## 1.4 提案手法の概要

本研究は，性格の異なるペルソナを持つ LoRA エージェントの社会が，議論を通じて問題を解き，その議論から学び，チームへの貢献で次世代の代表を選ぶ**進化的社会学習**を提案する（第3章）。手法は次の3つの要素からなる。

- **世代0のペルソナの選定**: 人間の問題解決チームの役割を分類した Belbin のチーム役割 [16, 17] の9役割を網羅的に候補とし，それぞれを推論の手続きとして操作化する。単独の評価，釣り合い型の実験計画によるチームの評価，別の問題での確認という3段階で，チームとしての成績に基づいて3体を選ぶ。既定の3体を置き換えるかどうかは，事前に定めた決定規則で決める。
- **議論**: 各エージェントが独立に解き（round0），自分の回答を保持したまま他者の回答を読んで解き直す（round1）。全員の答えが一致した問題では議論を省き，議論した問題では多数決で最終回答を決める。

- **進化的社会学習:** 各世代で、社会が新しい学習用の問題を議論する。各ペルソナの LoRA は、自分が正解した発話から学ぶ子（自己学習）と、自分が誤り他者が正解した問題で他者の解き方を学ぶ子（社会学習）を作り、両者を重みの空間で平均した子も加える。親と 3 体の子のうち、親の社会の中で全 7 連合の精度を実測して求めた厳密 Shapley 値が最大のものを、次世代の代表とする。

個体が議論で得た経験を学習して重みに取り込み、それが子に引き継がれる点はラマルクの的な継承 [18] に、集団がチームへの貢献で代表を選ぶ点はダーウィンの的な選抜に当たる。学習と選抜の二重の継承を持つ点で、本手法は文化的進化 [6] の構図に対応する。

検証では、選抜を行わない系統、単独の精度で選ぶ系統、ペルソナと議論を用いない単一モデルの自己学習 (RFT) [19, 20],  $SC@k$  を対照に置く (第 4 章)。

## 1.5 本研究の貢献

本研究の貢献は次のとおりである。

1. **ペルソナ LoRA 社会の進化的社会学習の定式化:** 議論の正解発話からの学習 (ラマルクの継承) と、全連合を実測した厳密 Shapley 値による選抜 (ダーウィンの選抜) を組み合わせた世代の手続きを定式化した。選抜の基準である Shapley 値が、単独の精度、2 体での協調、3 体のチームの精度を公理から導かれる重みで統合したものであることを示した。
2. **理論に基づくペルソナの選定の手続き:** チーム役割理論から網羅的に作った候補を推論の手続きとして操作化し、釣り合い型不完備ブロック計画と事前に定めた決定規則によって、チームとしての成績からペルソナを選ぶ手続きを示した。ペルソナの選び方を、直観ではなくデータと事前の規則で説明できるようにした。
3. **統制された比較による検証:** 選抜と最終評価の問題の分離、監査した採点器、固定した実行環境、事前に固定した統計の計画のもとで、選抜の有無、選抜の基準、単一モデルの自己学習、SC と比較した。

4. **予備研究の再検証:** 予備研究の生出力を監査し、採点器の欠陥が条件によって偏って効き、条件間の比較を歪めていたことを定量的に示した。マルチエージェント議論の評価において、採点器の監査と固定が必要であることの具体例である。

## 1.6 本論文の構成

第2章では、マルチエージェント議論と集約、ペルソナと役割、議論からの学習、進化的なモデル最適化、チームへの貢献の定量化、評価の信頼性に関する研究を整理し、本研究の位置づけを述べる。第3章では、世代0のペルソナの選定、議論のプロトコル、チームへの貢献、進化的社会学習の手続きを定式化する。第4章では、問題集合、採点、比較する系統、統計の計画、パイロットで決めた設計値を述べる。第5章では、予備研究の再検証と本実験の結果を報告する。第6章では結果を考察し、本研究の限界を述べる。第7章で結論と今後の課題を述べる。

## 第2章 関連研究

本章では、本研究に関係する研究を6つの系統に分けて整理する。2.1節ではマルチエージェント議論と回答の集約を、2.2節ではペルソナと役割の設計を、2.3節では議論や自己生成データからの学習を、2.4節では進化的なモデル最適化と重み合成の前提条件を、2.5節ではチームへの貢献の定量化を、2.6節では評価の信頼性を扱う。最後に2.7節で本研究の位置づけを述べる。

### 2.1 マルチエージェント議論と集約

#### 2.1.1 議論の原典と批判

マルチエージェント議論 (Multi-Agent Debate; MAD) の原典とされる Du ら [2] の手続きでは、複数の LLM インスタンスがまず独立に回答する。続くラウンドでは、各インスタンスが自分の前回の回答を会話の文脈に保持したまま他者の回答を読み、自分の回答を更新する。同研究は、この手続きによって数学の文章題や事実性の課題で単体より精度が上がることを示した。Liang ら [3] は、単一モデルの自己反省が初期解に固執する「思考の退化」を指摘し、異なる立場を与えたエージェント同士の議論がこれを避けるのに必要だと論じた。

その後、MAD の効果は計算量を揃えた比較で大きく縮むことが示された。Smit ら [8] は複数の MAD 戦略を同じ枠組みで比較し、複数の推論経路の多数決をとる Self-Consistency (SC) [7] などの単純な手法に、MAD が安定して勝てないことを示した。サンプリングと投票だけでも、エージェント数に応じて性能は伸びる [1]。Zhang ら [9] も既存研究の評価設計の弱さを指摘し、公平な条件では MAD の優位が小さいこと、モデルの異質性を活かすべきことを論じている。したがって MAD の効果は、生成回数を揃えた SC と比べて初めて評価できる。

小型モデルでは、単純な反復サンプリングの優位がさらに明確である。1.5B～7B のモデルと数学課題による 36 の比較では、同じトークン数の反復サンプリングより確実に良い手法は一つもなく、自己点検型の手法の 18 比較はすべて負けで

あった [11]. 7~8B 級の同一モデル 10 体による議論では、同調率が最大 85.5% に達した. チームの誰かが正解しているのに最終回答が誤る差 (oracle との差) は最大 32.3pt に開き、議論は孤立した自己訂正の 2.1~3.4 倍のトークンを消費した [12].

議論の利得の源泉は、回答の更新そのものより最終段の投票にあるとする分析もある. Choi ら [10] は、MAD の利得の大半が多数決で説明でき、議論による信念の更新は期待値の上で正答率を改善しないマルチンゲールになることを示した. Kaesberg ら [21] は、推論課題では投票型の決定方式が他の方式より 13.2% 高く、投票前の議論ラウンドを増やすと性能が下がることを報告している. MAD 戦略のサーベイ [22] は、分野の手法が「全結合・逐語共有・短い記憶・投票による決着」という狭い型に慣習的に収束していると指摘する.

### 2.1.2 同調と多様性

議論の主な失敗様式は同調である. Wynn ら [23] は、正答していたエージェントが他者の推論に引きずられて誤答へ移る現象を報告した. MachineSoM [24] は、性格と思考様式の異なるエージェントからなる「社会」に、同調や合意形成といった人間の社会心理学的な振る舞いが現れることを示した. 論理推論で条件を統制した Wu ら [25] は、個体の推論能力と集団の多様性が議論の成否を主に決め、多数派の圧力が独立な訂正を抑えると結論している. Okawa [26] は、同調性が臨界点を超えると集団が偏った合意へ相転移し、温度サンプリングのノイズがこれを増幅し、異質性が抑制することを示した. 自己組織化したチームが専門家と非専門家の意見を平均してしまい、最良の個人より最大 41.1% 劣るという報告もある [27].

議論に効く多様性は、性格よりも推論の手続きとモデルそのものの違いから生じる. DMAD [15] は、全員が同じ推論手法を使う議論は同じ誤りから抜け出せない (mental set) と指摘した. そして Chain-of-Thought, Step-Back, Program-of-Thought といった異なる推論手法を各エージェントに割り当て、全員が同じ手法の議論を上回った (GPT-4o-mini の集計値で 55.47 から 56.54). ReConcile [28] は、異なるモデル同士の議論で MATH を 8% 改善し、モデルの違いに由来する多様性が性能に決定的であると述べている. Zhu ら [29] は、初期回答の多様性を高める初期化と、確信度に条件づけた更新によって、議論が正解側へ流れることを示した.

ただし、多様性は品質を損なわない範囲でのみ価値を持つ. Li ら [30] は、異なるモデルを混ぜるより、最も強い単一モデルの複数の出力を集約する方が多くの

場合に優れることを示し、混合が有効なのは構成モデルの品質が拮抗する場合に限られると分析した。

### 2.1.3 回答の集約

最終回答は、多数決のほか、モデル自身に候補を比べさせる方法でも決められる。GenSelect[31]は、 $N$ 個の候補解をモデルに長い推論で比較させ、最良の1つを選ばせる。RSA[32]は、候補の部分集合を集約して改良した解を作る操作を再帰的に繰り返す。ただしRSAの原著でも、本研究と同じQwen3-4B-Instruct-2507によるSuperGPQAでは、予算を揃えた多数決(48.2)がRSA(10ステップで47.39)とベース(41.85)を上回った。著者らは、多肢選択形式では多数決が特に強いと述べている。

候補を見せて更新させる操作が効くかどうかは、候補に含まれる正解の数に依存する。Farmanfarmaian[33]は、Qwen3-4Bによる競技数学で、同じ予算で新しく解き直す対照と比べ、候補に正解が2つ以上あれば正答率が+0.290上がり、全員が誤っていれば−0.123下がることを示した。FortunaとBertalanic[34]は、誰か1人が正解すれば足りる課題では、チームを大きくすると潜在的な上限(oracle)が5~20pt増える一方、単純多数の投票は単体比で0.5pt以内しか回収しないことを報告した。同研究では複数ラウンドの改訂が大きく効き、その利得は相手が1体でも29体でもほぼ同じであった。

以上をまとめると、議論の効果は、生成回数を揃えたSCとの比較、同調の抑制、推論手続きの多様性、集約の設計の4点に左右される。本研究はこれらを前提条件として扱い、議論の手続きと集約を固定したうえで、生成回数を考慮したSCとの比較を評価に含める。

## 2.2 ペルソナと役割

### 2.2.1 ペルソナ付与の効果と限界

システムプロンプトでペルソナを与えても、客観的な課題の精度は上がらないことが多い。Zhengら[13]は、162種類の役割、4系列のモデル、2,410問の事実問題を調べ、ペルソナの効果はほぼランダムであり、最良のペルソナを自動で選ぶ方法もランダムな選択と大差ないことを示した。Kimら[14]は、Llama3で役割演

技のプロンプトが12データセット中7つで推論を劣化させたと報告している。専門家のペルソナは、好みや安全性への整合を改善する一方で、知識を問う課題の精度を損なう [35]。

小型のモデルでは、複雑なペルソナの指示そのものが機能しにくい。複数のペルソナによる自己協働 (Solo Performance Prompting) の効果は GPT-4 でのみ現れ、GPT-3.5-turbo や Llama2-13b-chat では現れなかった [36]。

性格の効果は、課題の構造と同調のしやすさを通じて現れる。Keluskar ら [37] は、協調性の低い性格が発話を大きく変えても、構造化された課題 (コーディング) の成果にはほとんど影響しないことを示した。重み空間で性格特性を扱った Persona Cartography [38] では、特性ごとの LoRA がほぼ加法的に合成でき、協調性 (agreeableness) の軸が追従 (sycophancy) を左右した。これらは、多肢選択や数学のような構造化された課題では、ペルソナを性格のラベルではなく推論の手続きとして定義すべきことを示唆する。

## 2.2.2 人間のチーム役割理論

人間の問題解決チームの役割を分類した代表的な理論に、Belbin のチーム役割がある [16, 17]。Belbin は、思考系の Plant, Monitor Evaluator, Specialist, 行動系の Shaper, Implementer, Completer Finisher, 対人系の Co-ordinator, Teamworker, Resource Investigator の 9 役割を定義し (Specialist は 1993 年に追加)、役割のそろったチームほど成果が高いと主張した。

ただし、Belbin の理論の実証的な妥当性には留保がある。Furnham ら [39] は、役割を測る自己評価尺度の信頼性係数が高くなく、因子構造も想定どおりに現れないことを示した。Senior [40] は 11 チームの調査で役割のバランスと成果の関連に部分的な支持を得ており、Aritzeta ら [41] は 43 の実証研究をもとに構成概念の妥当性を総括している。したがって Belbin の理論は、チームの成果を予測する法則としてではなく、人間のチームの役割を網羅的に列挙する分類として用いるのが妥当である。同様の分類として、de Bono の 6 つの思考モード [42] も、議論における役割付与の一類型として参照されている [22]。

個人の能力と集団の多様性の関係については、Hong と Page [43] が、多様な問題解決者の集団が個人成績上位者の集団を上回りうることを計算モデルで示した。候補者が多いと、上位者どうしが互いに似てしまうためである。この結果には Thompson [44] による数学的な批判と、Kuehn [45] による反論がある。単独の成績

が高い個体を集めてもチームとして最適とは限らないという問題提起は、チームへの貢献でメンバーを選ぶ本研究の動機の一つである。

### 2.2.3 役割の割り当てと選抜

LLM のチームでは、役割の割り当て方が結果を大きく左右する。Zhang ら [46] は、議論の各役割にどのモデルを置くかを候補どうしの予備的な議論で採点して決め、全役割に同じモデルを置く場合を最大 74.8%、ランダムな配置を最大 29.7% 上回った。DyLAN[47] は、予備試行で各エージェントの重要度 (Agent Importance Score) を集計して候補からチームを選び、MMLU では 7 体を 4 体に絞りつつ、科目によって最大 25% 改善した。

役割や構成そのものを探索する研究もある。EvoAgent[48] は、エージェントの設定を変異・交叉・選択で進化させて多エージェント系を自動生成する。MASS[49] は、プロンプトとトポロジーの両方が多エージェント系の性能を決めるとして、両者を段階的に最適化する。EvoMAS[50] は、多エージェント系の構成を、実行の記録に基づく変異と交叉で進化させる。これらはプロンプトや構成の空間での探索であり、モデルの重みは変えない。

役割を推論の手続きとして具体化するには、確立したプロンプト手法が使える。原理や定義へ一段抽象化してから解く Step-Back[51]、計画を立てて小問に分けて解く Plan-and-Solve[52] と Least-to-Most[53]、得た解を逆向きに検証する Self-Verification[54]、類似の問題を自ら想起して適用する Analogical prompting[55] などである。本研究は第 3 章で、Belbin の 9 役割をこれらの手続きに対応づけ、世代 0 のペルソナの候補とする。

## 2.3 議論と自己生成データからの学習

正解ラベルのある問題で、モデル自身が生成した推論のうち正解に至ったものを学習する自己学習は、単一モデルで確立している。STaR[19] は、正解した推論で微調整を繰り返し、解けなかった問題では正解をヒントとして与えて推論を作り直す (rationalization)。RFT[20] は、複数回サンプリングした推論のうち正解のものを集めて微調整し、互いに異なる推論経路が多いほど効果が大きいことを示した。

議論の相互作用から得たデータで、複数のモデルを別々に学習させる研究もある。Multiagent Finetuning[56] は、同じベースモデルから始めた複数のモデルを、モデル間の相互作用で生成したデータでそれぞれ別々に学習させた。その結果、推論の多様性を保ったまま、単一モデルの自己改善より多くの反復にわたって改善が続いた。MACA[57] は、議論の多数派と少数派の推論を区別するように事後学習し、単体の自己一貫性と議論の活用をともに高めた。CANON[58] は、多数派に到達した解を教師側だけが参照できる文脈として使うラベルなしの自己蒸留で、pass@1 を最大 12pt 改善した。Sirius[59] は、多エージェント系で成功に至った推論の軌跡を集め、各エージェントを自分の役割の成功軌跡で別々に学習させた。DTE[60] は、議論の推論と合意の答えを使って単一のモデルを正解ラベルなしで強化学習させ、Qwen2.5 の 1.5B～14B を含む 6 モデルで数学の精度を上げた。ただし 3B 未満のモデルでは、2 回目の反復で精度が下がった。

強化学習で議論への適応を学ぶ研究も増えている。MAPoRL[61] は、議論の協調を報酬として複数のモデルを共同で学習させる。SDRL[62] は、初期回答と、議論を条件にした回答を同時に最適化する。MADA-RL[63] は、1.5B モデルの LoRA を生成役と批評役に特化させ、批評役の利得を生成役の合意に対する反実仮想で定義することで、数学の精度を 39.9% から 41.9% に上げた。これらはいずれも勾配による学習であり、チームへの貢献で個体を選抜する集団の仕組みは持たない。

学習で獲得した形質を子へ受け渡す進化は、ラマルクの進化と呼ばれる。進化計算では、局所探索で改良した解を遺伝子型に書き戻す方式として、書き戻さない方式と対比して調べられてきた [18]。後者でも、学習は進化の探索空間の形を変えて進化を速める（ボールドウィン効果 [64]）。深層学習の Population Based Training[65] も、性能の低い個体に性能の高い個体の重みを複製させ、ハイパーパラメータに摂動を加えて学習を続けさせることで、学習と選抜を交互に行う。人間の文化は、遺伝ではなく他者からの社会学習によって世代を越えて伝わり、蓄積する [6]。強化学習のエージェントでも、社会学習と独立の学習を釣り合わせると世代を越えた蓄積が生じ、同じ累積経験を 1 世代で学んだエージェントを上回った [66]。LLM のエージェントの社会でも、資源の多い上位半分を残し、その戦略を文章で次世代に伝える選抜と伝達を 10 世代繰り返したところ、モデルによっては協力が世代とともに進化した。ただし結果はモデルと乱数 seed に強く依存し、伝達はプロンプトの文章で行われて重みは変わらない [67]。本研究の進化的社会学習は、議論で正解に至った自他の発話から学ぶ個体の学習と、チームへの貢献による選抜を組み合わせるものであり、これらの系譜に位置づけられる。

## 2.4 進化的モデル最適化

### 2.4.1 重み空間の集団最適化

進化計算を LLM の重みや合成レシピに適用する研究は、一つの系統をなしている。Akiba ら [68] は CMA-ES で既存モデルのマージレシピを探索し、LoraHub[69] は既存の LoRA の合成係数を勾配なしで最適化した。いずれも既存のモデルを素材とする 1 点の探索であり、集団の世代交代は行わない。

重み空間を集団で探索する研究も現れた。Model Swarms[70] は、LoRA の専門家を粒子とみなし、粒子群最適化 (PSO) で重み空間を探索する。GENOME (およびその拡張の GENOME+) [71] は、交叉・変異・選択・継承・アンサンブルからなる世代交代を LLM の重みに適用し、専門家の重みのランダムな線形結合で初期集団を多様化する。ただし適応度は個体の少数ショット精度である。EvoPref[72] は、LoRA の集団を NSGA-II で進化させ、多様な整合を探索する。PopuLoRA[73] は、共有ベース上の LoRA 集団を出題者と解答者の対戦で共進化させる。同研究は、テンソルの標準偏差の 0.1~0.15 倍の変異や外挿型の交叉を含む 17 種類の演算子を試し、子が短い学習で親の水準へ戻れない演算子を除いた。

大規模なモデルでも、勾配を使わない進化戦略 (ES) が有効な場合がある。ES at Scale[74] は、全パラメータに対する ES を 30 個体の摂動集団で行い、適応度を貪欲デコードで評価した。Countdown では Qwen2.5-7B の GRPO (57.5) を上回る 66.8 を得た。

チームとしての評価を重み集団の最適化に組み込んだ研究もある。Heterogeneous Swarms[75] は、複数の LLM をつなぐ有向非巡回グラフ (DAG) の役割と、各モデルの重みを PSO で交互に最適化する。各モデルの貢献を測る JFK-score は、最良の DAG の各位置にモデルをランダムに割り当てる試行を繰り返し、効用を登場頻度で重み付けして平均したもので、周辺貢献ではない。同研究は 15 のベースラインに対して 12 課題の平均で 18.5% の改善を報告したが、生成回数を揃えた SC との比較は含まない。AC/DC[76] は、タスクベクトルの内挿による交叉と特異値へのノイズによる変異でモデルの集団を作り、合成課題の集団と共進化させる。そのうえで、誰か 1 体が正解する割合 (カバレッジ) が最大となるチームを事後的に選んだ (3 体で専門家比 +1.34%)。品質多様性 (QD) [77] の枠組みで、重みのマージによる交叉と SVD による変異を行う CycleQD[78] もこの系統に属する。主要な研究の対比を表 2.1 に示す。

表 2.1: 重み空間の集団最適化に関する主要な研究の対比

| 研究                          | 最適化の対象                    | 探索の方式                            | 世代交代 | 適応度・貢献の測り方                                | 個体間の相互作用                       |
|-----------------------------|---------------------------|----------------------------------|------|-------------------------------------------|--------------------------------|
| 進化的モデル<br>マージ [68]          | マージレシビ                    | CMA-ES                           | なし   | 合成したモデル単体の性能                              | なし                             |
| Model<br>Swarms[70]         | LoRA 専門家の<br>重み           | PSO                              | なし   | 個体の効用                                     | なし                             |
| GENOME[71]                  | LLM の重み                   | GA (交叉・変異・<br>選択・継承)             | あり   | 個体の少数ショット<br>精度                           | アンサンブル<br>(投票)                 |
| PopuLoRA[73]                | LoRA の集団                  | 演算子と勾配学<br>習の併用                  | あり   | 対 戦 成 績<br>(TrueSkill)                    | 敵対的な自己<br>対戦                   |
| Heterogeneous<br>Swarms[75] | DAG の役割と<br>各モデルの重み       | PSO                              | なし   | JFK-score (ランダ<br>ムな割り当ての平<br>均効用)        | DAG 上のメッ<br>セージ伝達              |
| AC/DC[76]                   | モデルの集団<br>(課題の集団と<br>共進化) | 内挿による交叉,<br>特異値ノイズに<br>よる変異      | あり   | 新規性とスキル.<br>チームはカバレッ<br>ジで事後に選抜           | best-of-N<br>(judge による<br>選択) |
| <b>本研究</b>                  | 役割ごとのペル<br>ソナ LoRA        | 議論からの社会<br>学習 (勾配), 兄<br>弟交叉, 選抜 | あり   | <b>議論チームへの全<br/>連合実測の Shap-<br/>ley 値</b> | <b>協調的な議論</b>                  |

## 2.4.2 重み合成の前提条件

重みの交叉が意味を持つには、親どうしが損失地形の上で線形に接続されている必要がある。Model Soups[79] は、同じ事前学習モデルから微調整した複数のモデルの重みを平均すると精度が上がることを示した。Task Arithmetic[80] は、微調整による重みの差分  $\Delta W$  をタスクベクトルとみなし、その加減算で能力を合成できることを示した。TIES-Merging[81] は差分の符号の衝突を解消し、DARE[82] は差分の冗長性を利用して、合成時の干渉を減らす。体系的な整理は Yang ら [83] のサーベイにある。

LoRA[4] では、この前提が崩れやすい。KnOTS[84] は、LoRA で得た  $\Delta W$  はフル微調整に比べてモデル間の整合が低く、素朴な平均で性能が落ちることを示し、SVD で共通の基底に揃えてから合成する方法を提案した。CoTo[85] は、同じ課題を異なる乱数シードで独立に学習した 2 つの LoRA を重み空間で内挿すると、中点で通常の LoRA の精度が 39% まで落ちる (CoTo の学習法では 79%) ことを示した。一方、Seeded LoRA[86] は、共通の種となるアダプタから学習を始めると、その後の独立な学習が同じ部分空間に留まり、合成できるようになることを示した。

LoRA の低ランク因子  $A$ ,  $B$  を別々に補間する方式には、交差項の問題もある。

2つのアダプタ  $B_1A_1$ ,  $B_2A_2$  の因子を係数  $\alpha$  で別々に補間すると

$$\begin{aligned} & ((1 - \alpha)B_1 + \alpha B_2)((1 - \alpha)A_1 + \alpha A_2) \\ &= (1 - \alpha)^2 B_1A_1 + \alpha^2 B_2A_2 + \alpha(1 - \alpha)(B_1A_2 + B_2A_1) \end{aligned} \quad (2.1)$$

となり、意図した凸結合  $(1 - \alpha)B_1A_1 + \alpha B_2A_2$  とは一致せず、意味を持たない交差項  $B_1A_2$ ,  $B_2A_1$  が混入する．また  $BA = (BR)(R^{-1}A)$  という再パラメータ化の不定性があるため、因子空間での補間は分解の取り方に依存する．したがって LoRA の交叉は  $\Delta W$  の空間で定義し、必要に応じて SVD で低ランクに再分解するのが妥当である．本研究の兄弟交叉（第3章）は、同じ親から学習した2つの子の  $\Delta W$  を平均し、SVD で再分解する．共通の親から分岐した子どうしを合成する点でも、上記の前提に沿った設計である．

## 2.5 チームへの貢献の定量化

協力ゲーム理論の Shapley 値 [87] は、連合の価値関数  $v$  のもとで、各プレイヤーの貢献を、あらゆる部分連合への限界貢献の重み付き平均として配分する．効率性・対称性・ナルプレイヤー・線形性の公理を満たす唯一の配分であり、機械学習ではデータの価値評価 [88] をはじめ広く使われている [89]．計算には一般に  $2^n$  個の連合の評価が必要なため近似法が発達したが、3体のチームなら全7連合を実測でき、連合の列挙に伴う近似は要らない．ただし各連合の価値は有限個の問題での精度として推定されるため、推定の誤差は残る．

Banzhaf 値 [90] は、限界貢献を部分連合について等しい重みで平均する別の配分である．Wang と Jia [91] は、学習の確率性によって効用にノイズが乗ると Shapley 値や LOO による順位が不安定になり、準値 (semivalue) の中では Banzhaf 値が順位の頑健性 (safety margin) を最大にすることを示した．3体の全連合を実測すれば、両者は同じデータから追加の費用なしに計算できる．

個体の貢献を「その個体がない場合」との差で測る考え方は、進化計算とマルチエージェント学習に古くからある．Wolpert と Tumer [92] の集合知の枠組みや、Agogino と Tumer [93] の差分評価は、全体の評価と整合し、かつ各個体の寄与に敏感な評価関数を設計する．COMA [94] は、反実仮想のベースラインを方策勾配に取り入れた．集団の性能への周辺貢献を適応度とするスパイキングニューラルネットワークのアンサンブルの共進化が、単体の進化や事後的なアンサンブルより良い結果を得たという報告もある [95]．

役割ごとの部分集団を、他の部分集団の代表と組ませて評価する方法は、協調的共進化 [96] として確立している。ただし協力者の選び方は結果を左右する [97]。代表との組合せだけで評価すると、最適ではないがどの協力者とも無難に機能する個体が選ばれやすい相対的過汎化が起こりうる [98]。

LLM の多エージェント系でも、貢献を学習や選抜の信号にする研究が増えている。SHARP [99] はあるエージェントを除いた軌跡との報酬の差を、C3 [100] は1つの発話を差し替えて最後まで続けた反実仮想を、それぞれ強化学習の credit に用いる。Agents that Matter [101] は、除去に基づく帰属 (LOO) が組合せ的な方法と同等にボトルネックを特定できることを示し、貢献の低いエージェントの基盤モデルを差し替えて最大 17% 改善し、費用を 35% 削減した。MoCA [102] は、相補性に基づく貪欲な選択でチームを構成する。その原型は、モデルのライブラリから前向きに選択するアンサンブル選択 [103] にさかのぼる。

チームの利得を多様性の指標で代替することには限界がある。アンサンブルの誤差は、構成員の平均誤差から構成員間の不一致を引いた形に分解でき [104]、多様性は誤差の独立した次元として扱える [105]。しかし Kim [106] は、MMLU-Pro で 30 の LLM から作った 3 体の部分集合のうち、多数決が最強の構成員を上回ったのは 9.98% にすぎず、よく使われる多様性の指標が構成員の能力の代理になっていることを示した。連合ごとにチームの成果そのものを測る Shapley 値や Banzhaf 値は、この交絡を避けられる。

## 2.6 評価の信頼性

LLM の評価で条件間の差を検定するには、問題を単位とした対応のある比較と、問題間の相関を考慮した標準誤差が推奨されている [107]。NLP における有意性検定の使い分けは Dror ら [108] が整理しており、複数の比較には Holm の方法 [109] などで族全体の誤り率を制御する。

評価値の揺れには、問題の標本抽出によるノイズと、同じ問題での生成のノイズがある。Wang [110] は、後者が前者を通常上回り、同じ問題で予測を繰り返して平均すると検出力が大きく上がることを示した。

ノイズのある評価で候補を選ぶと、選ばれた候補の評価値は楽観的に偏る。進化計算では、ノイズのある適応度への対処として、再サンプリングや再評価などが整理されている [111, 112]。LLM の評価でも、適応的な調整の後に得た勝者の

スコアは過大になるため、候補を凍結し、選抜と評価に別のデータを使う必要がある [113]. 少数の問題による評価については、IRT で選んだ約 100 問と IRT による推定を組み合わせれば全体との誤差が約 2% に収まる [114]. ただしこの結果は、無作為に選んだ少数の問題には当てはまらない.

評価の環境も結果を変えうる. 同じモデルと同じプロンプトでも、GPU のアーキテクチャが違えば、浮動小数点演算の非結合性とカーネルの選択によって出力が変わる [115]. vLLM の推論設定の違いが精度にまで影響しうることも報告されている [116].

正誤の判定は、生成文から回答を取り出す抽出と、取り出した回答を正解と照合する判定の 2 段からなる. 数学の自由記述では、分数・根号・単位などの表記の揺れがあるため、文字列の一致ではなく記号的な同値判定が必要となり、Math-Verify[117] のような規則ベースの検証器が使われている. 抽出や判定の誤りが条件によって偏れば、条件間の比較そのものが歪む. したがって採点器は評価の前に固定し、条件ごとに誤りを監査する必要がある.

## 2.7 本研究の位置づけ

表 2.2 に、隣接する研究系統と本研究の差分を示す.

本研究を構成する要素の多くには先行例がある. チーム内の貢献で重み集団の最適化を導く考え方は Heterogeneous Swarms[75] が、議論の相互作用から得たデータで複数のモデルを別々に学習させる考え方は Multiagent Finetuning[56] が、すでに扱っている. 周辺貢献を共進化の適応度とする考え方は差分評価の系譜 [93, 95] に、貢献に基づくチームの選抜は DyLAN[47] や Agents that Matter[101] に先行例がある. 本研究は、これらの要素のいずれかを初めて提案するものではない.

一方、2026 年 9 月 30 日時点で著者が行った文献探索 (arXiv と主要な国際会議の論文を対象としたキーワード検索と、本章で挙げた文献の引用の追跡) の範囲では、次の 4 点を同時に満たす研究は見つからなかった.

- (i) 人間のチーム役割理論から網羅的に作った候補を推論の手続きとして操作化し、チームへの貢献で世代 0 のペルソナを選ぶこと.
- (ii) 議論で正解に至った自分と他者の発話から、各ペルソナの LoRA を親から継続して学習させること (ラマルクの継承).

表 2.2: 隣接する研究系統と本研究の位置づけ

| 系統                | 代表的な研究                                                                  | 更新する対象                | 最適化の信号                                  | 本研究との差分                |
|-------------------|-------------------------------------------------------------------------|-----------------------|-----------------------------------------|------------------------|
| 議論の手続き・集約の改良      | 文献 [2, 15, 31, 32]                                                      | 推論時の手続き               | —                                       | エージェント自体を学習・選抜しない      |
| 役割・構成の探索と選抜       | DyLAN[47],<br>EvoAgent[48],<br>MASS[49], 文献 [46]                        | プロンプト・構成・配置           | 貢献度・検証性能                                | 重みを学習しない               |
| 議論からの学習           | Multiagent<br>Finetuning[56],<br>MACA[57],<br>CANON[58],<br>MADA-RL[63] | 重み（勾配）                | 議論のデータ・合意・報酬                            | チームへの貢献による集団の選抜を持たない   |
| 重み集団の進化           | GENOME[71],<br>PopuLoRA[73],<br>Model Swarms[70]                        | 重み・LoRA の集団           | 個体の性能・対戦成績                              | 適応度がチームへの貢献でない         |
| チームの評価による重み集団の最適化 | Heterogeneous<br>Swarms[75],<br>AC/DC[76]                               | 役割と重み, モデルの集団         | JFK-score, カバレッジ                        | 周辺貢献で測らない. 議論を価値関数としない |
| 貢献に基づく強化学習        | SHARP[99],<br>C3[100]                                                   | 重み（勾配）                | LOO・反実仮想の credit                        | 集団の選抜でない               |
| <b>本研究</b>        | —                                                                       | <b>役割ごとのペルソナ LoRA</b> | <b>議論の正解発話（学習）と全連合実測の Shapley 値（選抜）</b> | —                      |

- (iii) 議論の多数決による精度を価値関数とし、全連合を実測した Shapley 値で次世代の代表を選ぶこと.
- (iv) 選抜なし、単独精度による選抜、ペルソナと議論を用いない単一モデルの自己学習、生成回数を考慮した SC を対照とし、事前に固定した統計計画で比較すること.

本研究の新規性の主張は、この組合せと、その統制された検証に限る.

評価の面では、2.1 節の批判を受けて、生成回数と出力トークン数を考慮した SC を基準に置く. また 2.6 節の知見に従い、選抜に使う問題と最終評価の問題を分け、問題をクラスタとした検定で結論を導く.

## 第3章 提案手法

本章では、役割の異なるペルソナを持つ LoRA エージェントの社会が、議論を通じて問題を解き、その議論から学び、チームへの貢献度に基づく選抜を経て世代を重ねる**進化的社会学習**を定式化する。

手法は二つの空間で働く。世代0では、チーム役割理論から作った候補の中から、チームとしての成績に基づいて3体のペルソナを選ぶ（**プロンプト空間**での選抜）。世代1以降では、各ペルソナの LoRA を社会の議論の記録から学習して子を作り、厳密 Shapley 値によるチーム貢献度で次世代の代表を選ぶ（**重み空間**での選抜）。どちらの段階でも、選抜の基準は「チームの一員として役に立つか」である。

3.1節で問題設定と記号を、3.2節で世代0のペルソナの選定を、3.3節で議論のプロトコルを、3.4節でチーム貢献度を、3.5節で進化的社会学習の1世代の手続きを述べ、3.6節で計算量を分析する。本章の記述は実装（src/evo4/と scripts/v4/）と一対一に対応する。手法の全体像を図 3.1 に示す。

### 3.1 問題設定と記号

#### 3.1.1 ベースモデルと LoRA 個体

凍結したベースモデル  $\mathcal{M}_{\theta_0}$ （本研究では Qwen3-4B-Instruct-2507[5]）を全エージェントで共有し、その上に付加する LoRA[4] アダプタを個体とする。対象モジュール  $m$ （各 Transformer 層の  $q$ ,  $k$ ,  $v$ ,  $o$  射影と, gate, up, down 射影）の重み  $W_0^{(m)} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$  に対し、LoRA は低ランクの行列の対  $B^{(m)} \in \mathbb{R}^{d_{\text{out}} \times r}$ ,  $A^{(m)} \in \mathbb{R}^{r \times d_{\text{in}}}$  による更新

$$W^{(m)} = W_0^{(m)} + \gamma \Delta W^{(m)}, \quad \Delta W^{(m)} = B^{(m)} A^{(m)}, \quad \gamma = \frac{\alpha_{\text{LoRA}}}{r} \quad (3.1)$$

を与える（本研究では  $r = 16$ ,  $\alpha_{\text{LoRA}} = 32$ ）。個体  $x$  の実効的な表現型は、全モジュールの更新行列  $\{\Delta W^{(m)}(x)\}_m$  である。LoRA を持たない個体（ベースモデル

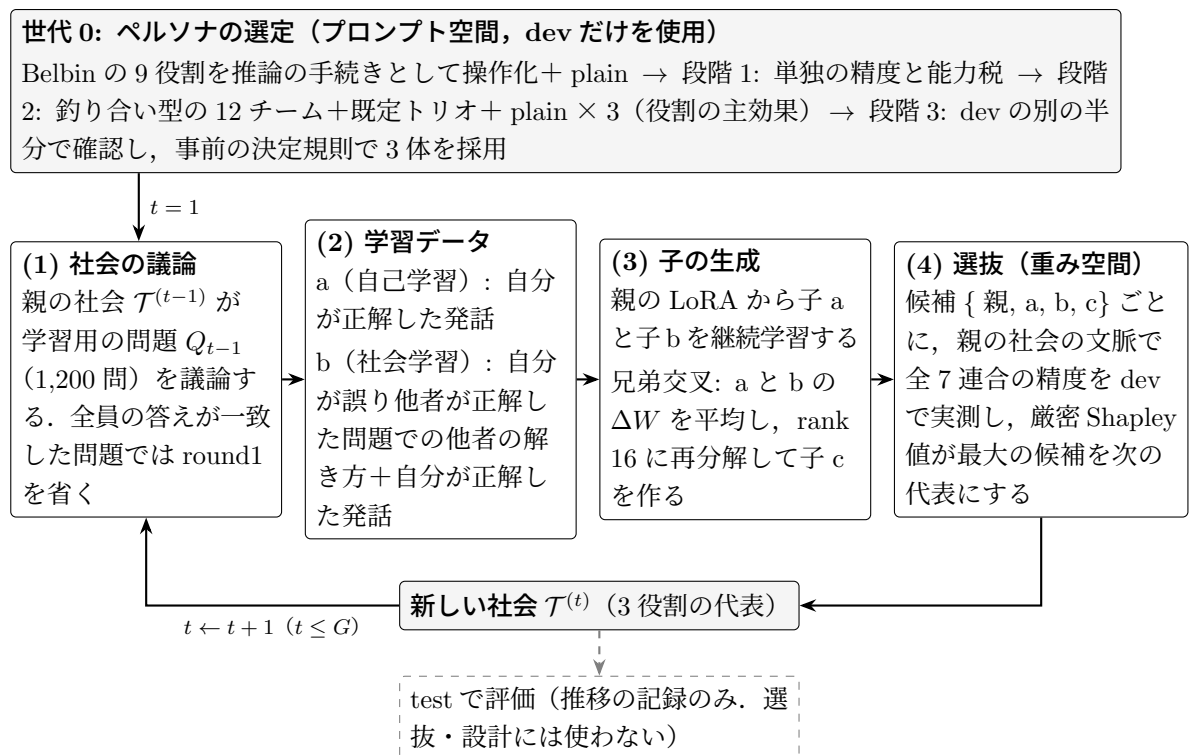


図 3.1: 進化的社会学習の全体像. 世代 0 ではプロンプト空間でペルソナを選び, 世代 1 以降は議論からの学習と, チームへの貢献による選抜を繰り返す.

そのもの) も許し, これを  $x = \emptyset$  と書く.

### 3.1.2 役割・エージェント・社会

役割の集合を  $\mathcal{R}$  ( $|\mathcal{R}| = 3$ ) とし, 各役割  $r$  に固有のペルソナ文  $\pi_r$  を割り当てる. ペルソナ文はシステムプロンプトに入れる 1~2 文の役割の記述である. エージェントは個体とペルソナ文の組  $a = (x, \pi_r)$  である. 役割とペルソナ文は世代 0 で選び (3.2 節), 以後のすべての世代で固定する.

世代  $t$  における役割  $r$  の代表を  $\rho_r^{(t)}$  と書き, 3 役割の代表からなるチーム  $\mathcal{T}^{(t)} = \{\rho_r^{(t)}\}_{r \in \mathcal{R}}$  を世代  $t$  の**社会**と呼ぶ. 世代 0 の代表は LoRA を持たない. 世代  $t \geq 1$  では, 役割ごとの候補の集合  $\mathcal{C}_r^{(t)}$  (3.5 節) から次の代表を選ぶ.

### 3.1.3 連合と特性関数

エージェントの集合 (連合)  $S$  が 3.3 節の議論のプロトコルで問題  $q$  を解いたときの最終回答を  $\mathcal{D}(S, q)$  と書く.  $|S| = 1$  のときは単独の回答である. 選抜に用いる問題集合 (dev) を  $D$ , そのうちベンチマーク  $k$  に属する問題の集合を  $D_k$  とする. 連合  $S$  の**特性関数**を, ベンチマークを等しく重み付けした dev の正答率 (macro 精度)

$$v(S) = \frac{1}{K} \sum_{k=1}^K \frac{1}{|D_k|} \sum_{(q, y^*) \in D_k} \mathbf{1}[\mathcal{D}(S, q) \equiv y^*], \quad v(\emptyset) = 0 \quad (3.2)$$

と定義する. ここで  $K = 3$  はベンチマークの数,  $y^*$  は正解,  $\equiv$  は第 4 章の採点規則による正誤判定である. macro 精度を用いるのは, 問題数の多いベンチマークに選抜が引きずられないようにするためである. 以上の記号を表 3.1 にまとめる.

## 3.2 世代 0: チーム役割理論に基づくペルソナの選定

### 3.2.1 候補の作り方

ペルソナの選び方はチームの成績を左右しうる設計変数だが, 恣意的に決められがちである. 2.2 節で見たとおり, 性格を表すラベルだけのペルソナは客観的な課題の精度をほとんど変えない [13, 14]. 一方, 議論では, エージェントごとに異

表 3.1: 記号表

| 記号                                | 意味                                                                      |
|-----------------------------------|-------------------------------------------------------------------------|
| $\mathcal{M}_{\theta_0}$          | 凍結したベースモデル (Qwen3-4B-Instruct-2507)                                     |
| $x, \Delta W^{(m)}(x)$            | LoRA 個体 (rank $r = 16$ ), モジュール $m$ の更新行列.<br>$x = \emptyset$ は LoRA なし |
| $\mathcal{R}, \pi_r$              | 役割の集合 (3 役割), 役割 $r$ のペルソナ文                                             |
| $a = (x, \pi_r)$                  | エージェント                                                                  |
| $\rho_r^{(t)}, \mathcal{T}^{(t)}$ | 世代 $t$ の役割 $r$ の代表, 世代 $t$ の社会 (代表 3 体のチーム)                             |
| $\mathcal{C}_r^{(t)}$             | 世代 $t$ の役割 $r$ の候補 (親, 子 a, 子 b, 子 c)                                   |
| $D, D_k, K$                       | 選抜に用いる問題集合 (dev), そのうちベンチマーク $k$ の問題, ベンチマークの数                          |
| $\mathcal{D}(S, q)$               | 連合 $S$ の問題 $q$ に対する最終回答                                                 |
| $v(S)$                            | 特性関数 (連合 $S$ の dev での macro 精度. $v(\emptyset) = 0$ )                    |
| $T(c)$                            | 候補 $c$ の代表チーム文脈 ( $c$ と, 他の 2 役割の現在の代表)                                 |
| $\phi_c, \beta_c$                 | 候補 $c$ の厳密 Shapley 値, Banzhaf 値                                         |
| $G$                               | 世代数                                                                     |

なる推論の手続きを割り当てると, 同じ手続きのエージェント同士の議論より精度が上がる [15]. そこで本研究は, 候補を次の手順で系統的に作る.

1. **理論による網羅:** 人間の問題解決チームの役割を分類した Belbin のチーム役割 [16, 17] の 9 役割を, 取捨選択せずすべて候補とする. 9 役割は思考系 (Plant, Monitor Evaluator, Specialist), 行動系 (Shaper, Implementer, Completer Finisher), 対人系 (Co-ordinator, Teamworker, Resource Investigator) の 3 群に分かれる. Belbin の理論はチームの成果を予測する法則としては実証的な留保があるため (2.2.2 項), ここでは役割を網羅的に列挙する分類としてだけ用いる.
2. **推論の手続きとしての操作化:** 各役割がチームで果たす機能を, 確立したプロンプト手法に対応する推論の手続きとして英語 1~2 文で記述する (表 3.2). 性格を表す語は最小限にとどめる.
3. **対照:** ペルソナ文を持たないエージェント (plain) を加える. さらに, 乱数 seed の系列だけが異なる plain の複製を 2 体加え, ペルソナの効果을「同じモデルの再サンプリング」と比べられるようにする.

表 3.2: ペルソナの候補 (Belbin の 9 役割の操作化. 英語のペルソナ文の全文は付録 A)

| Belbin の役割                    | 群  | 推論の手続き (ペルソナ文の要旨)                                     | 対応する手法         |
|-------------------------------|----|-------------------------------------------------------|----------------|
| Plant (創造者)                   | 思考 | 問題の解釈か解き方を 2 通り以上考え, 最も根拠のあるものを選ぶ                     | 解法の多様化 [15]    |
| Monitor Evaluator (監視評価者)     | 思考 | 選択肢や途中の主張を問題の条件と一つずつ照合し, 合わないものを消去してから答える             | 消去法・批判的評価      |
| Specialist (専門家)              | 思考 | 問題を支配する原理・定義・公式を先に述べ, それを段階的に適用する                     | Step-Back[51]  |
| Shaper (形づくる人)                | 行動 | 最も有力な答えにいったん決め, その最も弱い前提を攻撃して反証を試み, 反証に成功したときだけ答えを変える | 仮説と反証          |
| Implementer (実行者)             | 行動 | 最も標準的で直接的な方法を体系的に適用し, 結果を概算で検算する                      | 標準解法と検算        |
| Completer Finisher (完成者)      | 行動 | 解いた後, 答えを条件に代入し直すか別の方法で導き直して検証し, 誤りを直す                | 自己検証 [54]      |
| Co-ordinator (調整者)            | 対人 | 問題を短い小問の計画に分け, 順に解いて結果をまとめる                           | 計画と分割 [52, 53] |
| Teamworker (チームワーカー)          | 対人 | 他者が追って検証できるよう推論を平易に説明し, 問題の最もありそうな誤読を先に解消する           | 説明と誤読の解消       |
| Resource Investigator (資源探索者) | 対人 | 確信のある類題・例・事実を想起し, その解き方を本問に合わせて適用する                   | 類推 [55]        |
| plain (対照)                    | —  | ペルソナ文なし (回答書式の指示のみ)                                   | —              |

先行研究で当初想定していた3役割（批判的検証者，実務的意思決定者，発散的探索者）は，それぞれ Monitor Evaluator, Implementer, Plant に対応する．この3役割を**既定トリオ**と呼び，データに基づく選定で置き換えるかどうかの基準とする．既定トリオは Belbin の分類では思考系2体と行動系1体からなり，3群のそろった構成ではない．

9役割は，de Bono の6つの思考モード（Six Thinking Hats）[42] のすべてにも対応する役割を含む（対応は付録 A）．二つの独立した分類から見て，候補の集合に偏りが無いことを確認できる．

### 3.2.2 選定の手続き

dev を，ベンチマークごとに層化して固定 seed で半分ずつに分け，選抜に使う dev-A と確認に使う dev-B とする．選定は次の3段階で行い，dev だけを用いる（test は用いない）．選定の議論には 3.3 節のプロトコルを，ゲーティング（3.3.3 項）なしで用いる．

**段階1（単独の評価，dev-A）**．Belbin の9役割，plain，plain の複製2体の計12体について，単独の精度（macro）を測る．plain に対する差が  $-2\text{pt}$  以下で，かつその90%信頼区間（ベンチマーク内で問題を再抽出するブートストラップ）の上限が0未満の役割は，ペルソナが能力を損なう（**能力税**がある）とみなし，最終候補から除く．また，各役割と plain の回答の不一致率から，plain の複製同士の不一致率を引いた**超過不一致率**を記録する．これは，ペルソナが再サンプリング以上の回答の多様性を生むかどうかの指標である．

**段階2（チームの評価，dev-A）**．三つ組は  $\binom{9}{3} = 84$  通りあり，すべては評価できない．そこで，アフィン平面  $\text{AG}(2,3)$  の12本の直線を三つ組とする釣り合い型不完備ブロック計画を用いる．9役割を  $\mathbb{Z}_3^2$  の点  $(x,y)$  に対応させ， $x$  を群， $y$  を群内の番号とする．この計画には次の性質がある．

- 各役割はちょうど4チームに入り，どの2役割もちょうど1回だけ同じチームになる．
- $x$  が一定の3本の直線は同じ群の3役割からなる**同質型**のチームになり，残りの9本は3群から1体ずつの**バランス型**のチームになる．

この12チームに既定トリオと plain 3体のチームを加えた14チームを議論させ，

各チームの精度  $\text{acc}(T)$  を得る．計画内の 12 チームに加法モデル

$$\text{acc}(T) = \mu + \sum_{i \in T} \alpha_i + \varepsilon_T, \quad \sum_i \alpha_i = 0 \quad (3.3)$$

を最小二乗で当てはめ，役割  $i$  の主効果  $\alpha_i$  を推定する．この計画では推定値が

$$\hat{\alpha}_i = \frac{4}{3} (\bar{y}_i - \bar{y}) \quad (3.4)$$

と閉じた形になる．ここで  $\bar{y}_i$  は役割  $i$  を含む 4 チームの平均精度， $\bar{y}$  は 12 チームの平均精度である．主効果は，役割  $i$  をチームに入れたときの平均的な上積みを表す．Belbin の中心的な主張である「役割のそろったチームほど成果が高い」は，バランス型 9 チームと同質型 3 チームの平均精度の差として，チームのラベルの並べ替え検定 (20,000 回) で探索的に検定する．

**段階 3 (確認と決定, dev-B)**．次の最終候補を dev-B で議論させる．

- F0: 既定トリオ．
- F1: 段階 2 で観測した精度が最大のチーム．ただし段階 1 で除いた役割を含まず，plain 3 体のチームは除く．
- F2: 段階 1 で除かれなかった役割のうち，主効果  $\hat{\alpha}_i$  の上位 3 役割からなるチーム．
- F3: 単独の精度の上位 3 役割からなるチーム (単独の成績で選ぶ場合の対照)．
- C: plain 3 体のチーム (対照)．

F0～F3 のうち dev-B の macro 精度が最大のもの (同点なら F0) を  $F^*$  とする． **$F^*$  が F0 を +1.5pt 以上上回り，かつ対応のある差の 90%信頼区間の下限が 0 を超えるときだけ  $F^*$  を採用し，それ以外は F0 を採用する．**この決定規則は結果を見る前に定めた．この規則の下では，データが既定トリオより明確に良い組を示したときだけ置き換えが起こり，差を検出できない場合は理論上の既定に戻る．採用したチームについては，3.4 節の厳密 Shapley 値と Banzhaf 値を dev-B で求めて報告する．選定の結果と採用したトリオは第 5 章で報告する．

## 3.3 議論の Protokol

### 3.3.1 ラウンドの構造

議論は Du らのマルチエージェント議論 [2] の原型 (2.1.1 項) に従い, 更新のラウンドを 1 回持つ. ラウンド数を 1 とするのは, 推論課題ではラウンドを増やしても利得が小さいか逆効果になるという報告があるためである [10, 21].

- **round0 (独立回答)**: 各エージェントは, システムプロンプト (ペルソナ文と回答書式の指示) と問題文だけを入力として, 段階的な推論と最終回答を生成する. 最終行は ANSWER: <値> の書式とする.
- **round1 (更新)**: 各エージェントは, 自分の round0 の回答を会話履歴 (assistant の発話) として保持したまま, 次の user の発話で他のエージェントの round0 の回答を受け取り, 推論を見直した解答を生成する. 他者の回答は匿名化し (「Agent 1」「Agent 2」), 提示順を問題ごとに固定した seed でシャッフルする.

議論の流れを図 3.2 に示す.

round1 の指示文には, 中立版と条件付き版の二つを用意した. 中立版は「他者の推論と自分の解答を比べ, 要所を検算して, 自分と他者のどちらの誤りも指摘し, 更新した解答を示せ. 答えは維持しても変えてもよい」と指示する. 条件付き版は「自分の推論に具体的な誤りを見つけた場合にだけ答えを変え, その誤りを明示せよ. 見つからなければ, 他者と意見が違ってても答えを維持せよ」と指示する. 本研究は, dev-A での比較 (第 4 章) に基づいて中立版を用いる. プロンプトの全文は付録 A に示す.

自分の前回の回答を会話履歴として保持することは, 原型の手続きでは当然の前提である. 予備研究の実装 (第 5 章) はこれを欠いており, エージェントは自分の元の答えを見ずに他者の回答だけを読んで「維持するか変えるか」を判断していた. 本研究ではこの不整合を解消した.

各生成は温度付きのサンプリングで行う (設定は第 4 章). 乱数 seed は (問題, 生成 seed, 役割, ラウンド) から決定的に定め, 同じ役割の候補同士は同じ seed の系列を用いる. ただし推論エンジンの seed の指定は出力を再現しない (第 4 章) ため, seed は監査のための記録であり, 比較の分散を減らす手段としては期待しない.

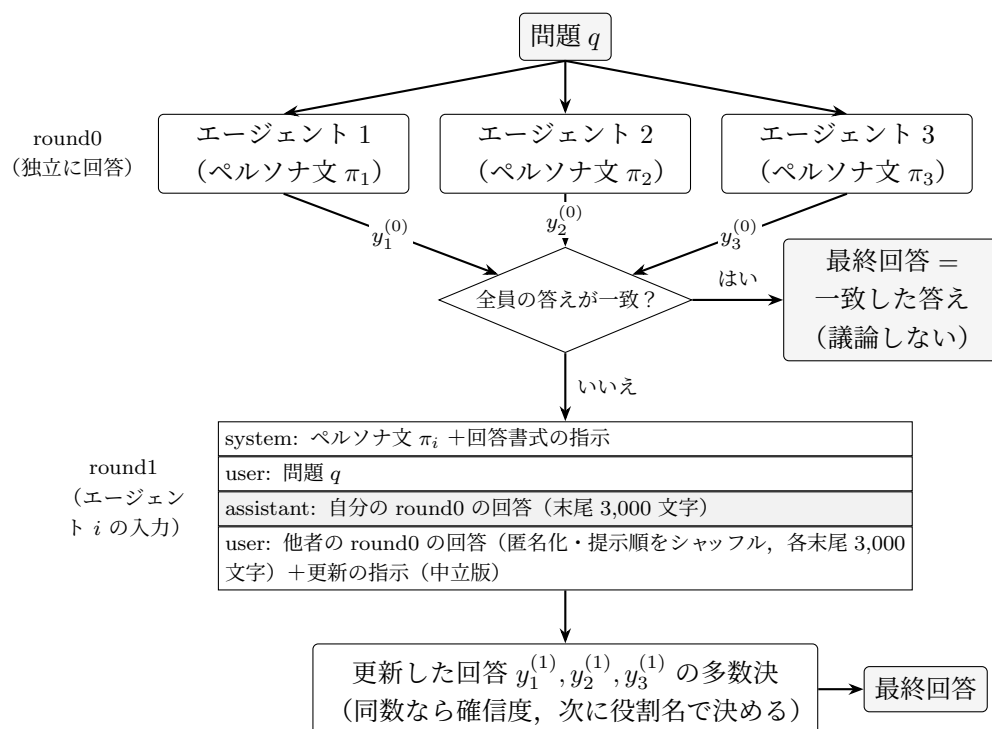


図 3.2: 議論のプロトコル. round0 の答えが全員一致した問題では議論を省く (3.3.3 項). round1 では, 自分の round0 の回答を会話履歴に保持したまま他者の回答を読む.

### 3.3.2 回答の切り詰め

round1 の入力自分と他者 2 体の回答の全文を含むため長くなる．入力が長いと、推論エンジンの KV キャッシュに同時に載る系列の数が減り、議論の生成が計算量の支配項になる（第 4 章）．そこで round1 では、自分の回答と他者の回答をそれぞれ**末尾 3,000 文字**に切り詰め、切り詰めた場合は先頭に前半を省いた旨の注記を付ける．結論と最後の計算は回答の末尾に現れるため、切り詰めても議論に必要な情報の多くは残ると考えられる．

予算の制約から、round1 の採用候補は切り詰め版の 2 種（中立版と条件付き版）に限り、dev-A で精度の高い方を採用する（中立版との差が 1pt 未満なら中立版とする）と事前に定めた．切り詰めない全文版は、切り詰めによる精度の損失を測る参照とした．結果は第 4 章に示す．

### 3.3.3 議論のゲーティング

連合の全員の round0 の回答が一致した問題では、round1 を生成せず、一致した回答をその連合の最終回答とする．議論するのは一致しない問題だけである．回答の一致は、抽出した回答の文字列の一致で判定する（数学の問題では、表記の異なる同値な回答は一致とみなさない）．

ゲーティングの根拠は、round0 で多数派だったエージェントが議論の後も答えをほとんど変えないことにある（第 4 章）．全員が一致した問題で議論しても最終回答はほぼ変わらず、計算だけがかかる．dev-A の予備比較では、ゲーティングによる精度の低下は見られず、round1 の呼び出しは約 6 割減った（第 4 章）．ゲーティングは進化の各系統における議論、選抜、評価に適用し、世代 0 の選定と 7 月のチームの評価（第 4 章）には適用しない．

### 3.3.4 集約

連合  $S$  の最終回答  $D(S, q)$  は次のように定める．

- $|S| = 1$  なら、round0 の回答とする．
- $|S| \geq 2$  でゲーティングにより議論しなかった場合は、一致した round0 の回答とする．

- 議論した場合は、全員の round1 の回答の多数決とする．回答を抽出できない発言は無効票として除く．

同数の場合は、各回答に投票したエージェントの発言の平均トークン対数確率（確信度）の最大値が大きい回答を選ぶ．それも等しければ、投票したエージェントの役割名の辞書順、最後に回答の文字列の順で決める．乱数は使わず、同じ生成結果からは同じ最終回答が決まる．2体の連合で意見が割れた場合は、この規則により確信度の高い方の回答が選ばれる．

判定者（judge）のエージェントによる集約は採らない．推論課題では合意形成より多数決が一貫して優れること [21]，議論の利得の大半が多数決で説明されること [10]，議論の最終判断を担う者の能力が結果を左右すること [118] が理由である．

## 3.4 チーム貢献度

### 3.4.1 代表チーム文脈における厳密 Shapley 値

役割  $r$  の候補  $c$  の貢献は、他の 2 役割の現在の代表と組ませた**代表チーム文脈**

$$T(c) = \{c\} \cup \{\rho_{r'}^{(t-1)} \mid r' \in \mathcal{R}, r' \neq r\} \quad (3.5)$$

で測る．これは協調的共進化 [96] における、協力者を固定した部分解の評価に当たる． $T(c)$  上の厳密 Shapley 値 [87] は

$$\phi_c = \sum_{S \subseteq T(c) \setminus \{c\}} \frac{|S|!(n - |S| - 1)!}{n!} \left( v(S \cup \{c\}) - v(S) \right), \quad n = 3 \quad (3.6)$$

である．他の 2 役割の代表を  $\rho_1, \rho_2$  と書いて展開すると

$$\begin{aligned} \phi_c = & \frac{1}{3}v(\{c\}) + \frac{1}{6}\left[v(\{c, \rho_1\}) - v(\{\rho_1\})\right] + \frac{1}{6}\left[v(\{c, \rho_2\}) - v(\{\rho_2\})\right] \\ & + \frac{1}{3}\left[v(T(c)) - v(\{\rho_1, \rho_2\})\right] \end{aligned} \quad (3.7)$$

となる．2体の連合  $\{c, \rho_1\}$  の値は、 $c$  と  $\rho_1$  の2体だけで3.3節の議論を行ったときの精度である．必要な特性関数の値は3体の全非空連合の  $2^3 - 1 = 7$  個であり、本研究はこれを**すべて実測**する．したがって連合の列挙に近似はない．ただし各  $v(S)$  は dev という有限の標本での推定値であり、推定の誤差を含む．

Shapley 値を採るのは、効率性・対称性・ナルプレイヤー・線形性の公理を同時に満たす唯一の配分であるためである [87, 88]．Leave-One-Out 型の寄与  $v(T) - v(T \setminus \{c\})$

は、同じ振る舞いの個体が連合に共存すると双方の寄与を 0 とする欠陥を持つ [88]. 同じ親から学習した子は振る舞いが近くなりやすいので、この欠陥は選抜を歪めうる.

代表との組合せだけで評価すると、最適ではないがどの協力者とも無難に機能する個体を選ばれやすくなる（相対的過汎化，2.5 節）. 本研究は、協力者を前世代の代表に固定する単純な方式を採り、この偏りは限界として扱う.

### 3.4.2 Shapley 値が統合するもの

同じ役割の候補同士では、文脈だけからなる 3 つの連合の値、すなわち  $v(\{\rho_1\})$ ,  $v(\{\rho_2\})$ ,  $v(\{\rho_1, \rho_2\})$  が共通である. したがって

$$\phi_c = \frac{1}{3}v(\{c\}) + \frac{1}{6}\left[v(\{c, \rho_1\}) + v(\{c, \rho_2\})\right] + \frac{1}{3}v(T(c)) - \kappa_r \quad (3.8)$$

と書け、候補の順位は第 1～3 項で決まる. ここで  $\kappa_r$  は候補に依存しない定数である. 重みの和は  $1/3 + 1/6 + 1/6 + 1/3 = 1$  であり、Shapley 値は、候補を含む 4 つの連合の精度の加重平均から定数を引いたものに等しい. つまり Shapley 値による選抜は、単独の精度（第 1 項）、2 体での協調（第 2 項）、3 体のチームとしての精度（第 3 項）を、公理から導かれる重みで一つの値に統合している. 単独の精度だけで選ぶ対照（3.5.5 項の系統 A1）は第 1 項だけを見ることに当たり、両者の比較が「チームへの貢献で選ぶこと」の効果の検証になる.

### 3.4.3 Banzhaf 値

同じ 7 連合からは、Banzhaf 値 [90]

$$\beta_c = \frac{1}{2^{n-1}} \sum_{S \subseteq T(c) \setminus \{c\}} \left( v(S \cup \{c\}) - v(S) \right) \quad (3.9)$$

も追加の推論なしに計算できる. Banzhaf 値は、特性関数に雑音があるときに順位が安定しやすい [91]. 選抜には Shapley 値を用い、Banzhaf 値は順位の頑健性を確かめるために記録する.

## 3.5 進化的社会学習

世代  $t$  ( $t \geq 1$ ) の手続きは、親である世代  $t-1$  の社会  $\mathcal{T}^{(t-1)}$  の議論から学習データを作り、役割ごとに子を学習・交叉し、チーム貢献度で次の代表を選ぶ、という流れである (Algorithm 1)。個体は議論で得た経験を学習して重みに取り込み、それが子に引き継がれる。これは獲得形質の継承であり、ラマルクの進化 [18] に当たる。学習（個体が他者から学ぶこと）と選抜（集団が代表を選ぶこと）の二重の継承を持つ点で、文化的進化 [6] の構図とも対応する。

### 3.5.1 社会の議論

世代  $t$  では、学習用の問題集合（世代ごとに新しい 1,200 問、第 4 章）で親の社会  $\mathcal{T}^{(t-1)}$  が議論する。プロトコルは 3.3 節と同じで、全員の答えが一致した問題では round1 を生成しない。

### 3.5.2 学習データ

学習データは、正解が分かる学習用の問題で、正解に至った発話だけから作る。用いるのは生成が完了した（長さの上限で打ち切られていない）発話に限る。役割  $r$  の代表  $\rho_r$  について、二つのデータを作る。

- **a (自己学習)** :  $\rho_r$  自身の、正解した round0 の発話と、正解した round1 の発話。round1 の例は、round1 の入力（自分の前回の回答と、切り詰めた他者の回答）をそのまま含む。ゲーティングの下では、round1 の例は意見が割れた問題からだけ得られる。
- **b (社会学習)** :  $\rho_r$  の round0 が正解でなく、他者の round0 が正解した問題について、正解した他者の発話を、 $\rho_r$  自身のシステムプロンプト（自分のペルソナ文）の下での回答として学ぶ例。正解した他者が 2 体いる場合は、役割の固定順で先の 1 体の発話を用い、1 問につき 1 例とする。他者から学ぶ例はすべて含め、上限に満たない分を a の例から無作為に補う。

どちらも上限は 1,000 例とし、a は上限を超えれば無作為に抽出する。a は「自分の成功体験から学ぶ」ことに、b は「他者の成功から学ぶ」ことに当たり、人間社会における個人学習と社会学習の区別に対応する。

### 3.5.3 LoRA の学習

子は親の LoRA から継続して学習する．世代 1 の親は LoRA を持たないので，新しい LoRA を初期化する．ベースモデルは bf16 のまま量子化せずに用いる．学習と推論で同じ精度のベースモデルを使うためである．

損失は，各例の最後の assistant の発話（とその終端トークン）のトークンだけに掛ける．プロンプトの部分（ペルソナ文，問題，他者の回答）には掛けない．例  $e$  のトークン列を  $u_{1:L_e}$ ，損失を掛ける位置の集合を  $\Lambda_e$  とする．ラベルのトークンの累計が  $N_{\text{tok}}$  に達するまで例を積み上げたものを 1 回の更新の単位  $\mathcal{B}$  とし，その損失を

$$\mathcal{L}_{\mathcal{B}} = -\frac{1}{N_{\text{tok}}} \sum_{e \in \mathcal{B}} \sum_{\ell \in \Lambda_e} \log p_{\theta}(u_{\ell} \mid u_{<\ell}), \quad N_{\text{tok}} = 32,768 \quad (3.10)$$

とする．出力層の計算は  $\Lambda_e$  の位置だけで行い，長いプロンプトでもメモリを節約する．学習率は新しい LoRA で  $1 \times 10^{-4}$ ，継続学習で  $5 \times 10^{-5}$  とし，1 エポック学習する（他の設定は第 4 章）．

### 3.5.4 兄弟交叉

同じ親から学習した二つの子  $a$  と  $b$  を  $\Delta W$  の空間で交叉して，第三の子  $c$  を作る．LoRA の因子  $A$ ,  $B$  を別々に補間すると，意図した補間と一致せず，意味を持たない交差項が混入する（2.4.2 項，[84]）．そこで補間を  $\Delta W$  の空間で行う．各モジュール  $m$  について

$$\Delta W_c^{(m)} = (1 - \alpha) B_a^{(m)} A_a^{(m)} + \alpha B_b^{(m)} A_b^{(m)}, \quad \alpha = 0.5 \quad (3.11)$$

を作り，ランダム化 SVD[119] で上位  $r$  成分  $U_r \Sigma_r V_r^{\top}$  を求めて

$$B_c^{(m)} = U_r \Sigma_r^{1/2}, \quad A_c^{(m)} = \Sigma_r^{1/2} V_r^{\top} \quad (3.12)$$

とする．子  $c$  の更新  $B_c A_c$  は，補間した  $\Delta W_c$  の rank- $r$  での最良近似（ランダム化 SVD の精度の範囲で）である． $a$  と  $b$  は同じ  $\gamma$  を持つので， $\Delta W$  の補間は実効的な更新  $\gamma \Delta W$  の補間と等しい．

交叉の相手を兄弟に限るのは，独立に学習した LoRA 同士の平均は性能を損ないやすいからである [85]．共通の種となるアダプタから分岐した LoRA は同じ部分空間に留まり，合成できるようになる [86]．同じ事前学習モデルから微調整した重みの平均が精度を上げる Model Soups[79] と同じく，兄弟の平均は，両者の学習内容を併せ持つ中間の個体を与えると期待できる．

### 3.5.5 選抜

役割  $r$  の候補を

$$\mathcal{C}_r^{(t)} = \{\rho_r^{(t-1)}, a_r^{(t)}, b_r^{(t)}, c_r^{(t)}\} \quad (3.13)$$

とする. 親  $\rho_r^{(t-1)}$  を候補に残すのは, 子がすべて親より悪い場合に親を残せるようにするため (エリート保存) である.

提案手法 (系統 S) は, 親の社会の文脈 (式 (3.5)) で各候補の厳密 Shapley 値を  $\text{dev}$  で計算し, 役割ごとに値が最大の候補を次の代表とする.

$$\rho_r^{(t)} = \arg \max_{c \in \mathcal{C}_r^{(t)}} \phi_c \quad (3.14)$$

3 役割の代表は, 同じ文脈 (親の社会) の下で同時に選ぶ. 同値の場合は, 個体名の辞書順で後のものを選ぶ (決定的な規則).

比較のため, 同じ候補から単独の精度  $v(\{c\})$  が最大のものを選ぶ系統 A1 と, 選抜を行わず常に社会学習の子  $b_r^{(t)}$  を代表とする系統 N を定義する (第 4 章).

### 3.5.6 アルゴリズム

以上を Algorithm 1 にまとめる.

## 3.6 計算量の分析

本手法の費用の大部分は LLM の生成が占め, なかでも他者の回答を入力に含む round1 の生成が重い.

**生成の共有.** round0 の出力は, エージェントと問題の組ごとに一度だけ生成して保存し, そのエージェントを含むすべての連合で共有する. round1 の出力は (エージェント, 相手の集合, 問題) の組ごとに保存する. 同じ呼び出しを二度生成することはない.

**選抜の費用.** 代表チーム文脈の 7 連合のうち, 文脈だけからなる 3 連合 ( $\{\rho_1\}$ ,  $\{\rho_2\}$ ,  $\{\rho_1, \rho_2\}$ ) は, 同じ役割の全候補で共有される. 新しい候補 1 体に固有の呼び出しは,  $\text{dev}$  の 1 問あたり, round0 が 1 回, 2 体の連合  $\{c, \rho_1\}$  と  $\{c, \rho_2\}$  の round1 が各 2 回, 3 体の連合  $T(c)$  の round1 が 3 回である. 新しい候補の数を  $m$  (3 役割

---

**Algorithm 1** 進化的社会学習（系統 S）

---

**入力:** 世代 0 の社会  $\mathcal{T}^{(0)} = \{(\emptyset, \pi_r)\}_{r \in \mathcal{R}}$ , dev  $D$ , 学習用の問題集合  $Q_0, \dots, Q_{G-1}$ , 世代数  $G$

**出力:** 最終世代の社会  $\mathcal{T}^{(G)}$

```
1: for  $t = 1, \dots, G$  do
2:    $\mathcal{T}^{(t-1)}$  が  $Q_{t-1}$  で議論する（ゲーティングあり） ▷ 3.5.1 項
3:   for  $r \in \mathcal{R}$  do
4:     正解に至った発話から  $\text{Data}_a$ （自己学習）と  $\text{Data}_b$ （社会学習）を作る ▷ 3.5.2 項
5:      $a_r^{(t)} \leftarrow \text{Train}(\rho_r^{(t-1)}, \text{Data}_a)$ ,  $b_r^{(t)} \leftarrow \text{Train}(\rho_r^{(t-1)}, \text{Data}_b)$  ▷ 継続学習
6:      $c_r^{(t)} \leftarrow \text{DeltaBlendSVD}(a_r^{(t)}, b_r^{(t)}; \alpha = 0.5)$  ▷ 兄弟交叉
7:      $\mathcal{C}_r^{(t)} \leftarrow \{\rho_r^{(t-1)}, a_r^{(t)}, b_r^{(t)}, c_r^{(t)}\}$ 
8:   end for
9:   for  $r \in \mathcal{R}$ ,  $c \in \mathcal{C}_r^{(t)}$  do
10:     $T(c) \leftarrow \{c\} \cup \{\rho_{r'}^{(t-1)} \mid r' \neq r\}$ 
11:     $T(c)$  の全 7 連合を  $D$  で議論させ,  $v(S)$  を得る（round0 は個体ごとに共有, ゲーティングあり）
12:     $\phi_c \leftarrow \text{Shapley}(c; v)$ ,  $\beta_c \leftarrow \text{Banzhaf}(c; v)$  ▷ 式 (3.6), (3.9)
13:   end for
14:   for  $r \in \mathcal{R}$  do
15:     $\rho_r^{(t)} \leftarrow \arg \max_{c \in \mathcal{C}_r^{(t)}} \phi_c$  ▷ 選抜
16:   end for
17:    $\mathcal{T}^{(t)} \leftarrow \{\rho_r^{(t)}\}_{r \in \mathcal{R}}$  を test で評価する ▷ 推移の記録. 選抜には使わない
18: end for
19: return  $\mathcal{T}^{(G)}$ 
```

---

で子が3体ずつなら  $m = 9$ ), dev の問題数を  $|D|$  とすると, 新しい候補に固有の round1 は

$$N_{r1} \leq m(2 + 2 + 3)|D| = 7m|D| \quad (3.15)$$

回以下である. これに, 親の社会の2体の連合と3体の連合の round1 (最大  $9|D|$  回) が加わるが, 前世代で計算済みのものは再利用する.

**ゲーティングの効果.** ゲーティングの下では, 連合  $S$  の round1 は, その連合の round0 が一致しない問題でだけ必要になる. 連合  $S$  で全員の回答が一致する問題の割合を  $u_S$  とすると,

$$\mathbb{E}[N_{r1}] = \sum_S |S|(1 - u_S)|D| \quad (3.16)$$

となる (和は新しい候補に固有の2体と3体の連合について取る). 3体のチームで round0 が全員一致する問題の割合は, dev-A の既定トリオで約6割だった (第4章). 2体の連合の一致率は, その2体を含む3体のチームの一致率以上であるため, round1 の呼び出しは少なくともその分だけ減る.

**学習と交叉の費用.** 1世代で学習する子は, 3役割について a と b の計6体であり, それぞれ1,000例以下を1エポック学習する. 交叉の子 c は学習を伴わず, ランダム化 SVD による重みの操作だけで作れる.

## 第4章 実験設定

本章では、第3章の手法を検証する実験の設定を述べる。4.1節でモデルと実行環境を、4.2節でベンチマークと問題集合を、4.3節で採点を、4.4節で系統と比較条件を、4.5節で評価指標と統計を、4.6節でハイパーパラメータとパイロットで決めた設計値を、4.7節で計算資源と予算を述べる。結果は第5章で報告する。

問題集合、主要な評価指標と検定、中止規則は、本実験の結果を見る前に研究設計の文書として固定し、2026年9月30日にリポジトリに記録した<sup>1</sup>。その後の変更は理由と日時を記録しており、4.6.3項にまとめる。

### 4.1 モデルと実行環境

#### 4.1.1 推論

ベースモデルは、指示調整済みの約40億パラメータのモデルQwen3-4B-Instruct-2507[5]である。推論にはvLLM 0.8.5[120]のOpenAI互換サーバを用いる。ベースモデルをbf16で読み込み、複数のLoRA（最大12個、rank 16まで）を同時に載せ、個体ごとに切り替えて生成する。LoRAは実行中に動的に読み込む。

生成の設定は全条件・全ラウンドで共通とし、温度0.7、top-p 0.8、top-k 20、出力の上限8,192トークンとする（表4.3）。温度とtop-p、top-kの値はモデルの推奨設定に合わせた。各トークンの対数確率も取得し、発話の平均トークン対数確率を集約の同点処理（3.3.4項）の確信度に用いる。

出力の上限に達した生成は、その時点の出力のまま採点する。途中で打ち切る規則を設けると条件間で扱いが変わりうるため、上限は全条件で共通とした。例外として、同じ文字列が周期的に繰り返す完全な反復に陥った生成だけは、生成中に検出して打ち切る。具体的には、生成した400文字ごとに末尾を調べ、同じ

---

<sup>1</sup>設計文書 docs/research.design.v4.md, コミット 216fc89 (ブランチ v4-preregistration)。

文字列が5回以上かつ合計1,500文字以上続いていれば打ち切る．完全一致の周期性だけを見る保守的な判定であり，正常な推論文を誤って打ち切りにくい．

### 4.1.2 学習

LoRA の学習には，`peft 0.15.2` と `transformers 4.51.3` (PyTorch 2.6.0) を用いる．ベースモデルは `bf16` で読み込み，注意機構には PyTorch の `scaled dot-product attention` を，メモリの節約に勾配チェックポイントを用いる．学習と推論は同じジョブの中で交互に行い，学習の間は推論サーバを止めて GPU を学習に充てる．

### 4.1.3 実行環境の固定と非決定性

すべての実験は，`vLLM 0.8.5` の公式イメージに学習用のパッケージを加えた一つのコンテナイメージで行う．イメージはダイジェスト (`sha256:41ab7010...`) で指定し，実験期間中は作り直さない．推論経路のパッケージの版は制約ファイルで固定し，追加のインストールがそれらを変えるとビルドが失敗するようにした．コードはイメージに含めず，各ジョブが起動時にクラウドストレージからコードの版ごとのスナップショットを取得する．予備研究では，実験の途中で環境を作り直したため，測定値の差が環境によるものかどうかを切り分けられなかった（第5章）．環境を一つに固定することで，この種の交絡を排除する．

`vLLM` の生成は，同じ `seed` を指定しても出力が再現しない（予備研究で確認した．第5章）．GPU 上の浮動小数点演算は結合的でなく，計算の順序によって結果が変わるためである [115]．そこで，すべての呼び出しを（ラウンド，エージェント，相手の集合，問題，生成 `seed`）からなる一意のキーで一度だけ生成して保存し，再生成しない．採点と統計は保存した出力に対してオフラインで行うため，生成は再現しなくても分析は再現できる．

## 4.2 ベンチマークと問題集合

### 4.2.1 ベンチマーク

性質の異なる3つのベンチマークを用いる．

- **MMLU-Pro**[121]: 多分野の知識と推論を問う多肢選択問題. 選択肢は最大 10 個 (A~J) である.
- **SuperGPQA**[122]: 大学院水準の専門知識を問う多分野の多肢選択問題. 選択肢は最大 10 個である.
- **MATH**[123]: 競技数学の自由記述問題. 難易度は Level 1~5 に分かれる. 評価には難度の高い Level 4 と 5 の問題だけを用いる.

多肢選択の問題は, 問題文の後に「A. 選択肢」の形式で選択肢を並べて提示する. 予備研究で精度がほぼ飽和していた GSM8K[124] は用いない.

#### 4.2.2 問題集合

問題集合を表 4.1 に示す. すべての集合は構築スクリプトで固定 seed から作り, JSONL ファイルとその SHA-256 ハッシュを記録した. 集合間に重複がないことは構築時に検査した.

表 4.1: 問題集合

| 集合                | MMLU-Pro | SuperGPQA | MATH  | 計       | 出典と用途                                                                            |
|-------------------|----------|-----------|-------|---------|----------------------------------------------------------------------------------|
| test              | 500      | 500       | 300   | 1,300   | 予備研究で未使用の問題. MATH は test 分割全体の Level 4-5 から MATH-500[125] の収録分を除いた問題. 最終評価だけに用いる |
| dev               | 150      | 150       | 100   | 400     | 予備研究で使用済みで, ベースモデルの正答率が既知の問題から層化抽出. MATH は MATH-500 の Level 4-5. 進化の選抜に用いる       |
| dev-A             | 75       | 75        | 50    | 200     | dev の半分. 世代 0 の選定とプロトコルの比較に用いる                                                   |
| dev-B             | 75       | 75        | 50    | 200     | dev の残り半分. 世代 0 の選定の確認に用いる                                                       |
| $Q_0, \dots, Q_5$ | 各 400    | 各 400     | 各 400 | 各 1,200 | 社会学習のデータ生成に用いる学習用の問題. MATH は train 分割の Level 3-5                                 |

test は, 予備研究で一度も使っていない問題から無作為に抽出した. test は最終評価だけに用い, 選抜や設計の判断には一切用いない. 適応的な調整に使ったデータで勝者を評価すると, そのスコアは過大になるためである [113].

**dev** は、予備研究で使用済みの問題から、ベースモデルの正答率  $p$  に基づいて層化して抽出した。  $p$  は、予備研究で保存したベースモデルの生出力（1問あたり複数の標本）を、本研究の採点器（4.3 節）で採点し直して求めた。各ベンチマークで、  $0 < p < 1$  の問題を 70%、  $p = 1$  の問題を 30%とし、  $p = 0$  の問題は含めない。  $0 < p < 1$  の問題は候補間の差を識別しやすい。  $p = 1$  の問題は、学習による能力の劣化（ベースモデルなら解ける問題を落とすこと）を検出するために含める。 dev における  $p$  の平均は、MMLU-Pro 0.638, SuperGPQA 0.666, MATH 0.853 である。

dev は、ベンチマークごとに層化して固定 seed で半分ずつに分け、dev-A と dev-B とした。 dev-A と dev-B は世代 0 の選定（3.2 節）に、dev 全体は進化の選抜に用いる。

**学習用の問題**  $Q_0, \dots, Q_5$  は、test, dev, 予備研究で使用済みの問題のいずれとも重ならない。MMLU-Pro は test 分割の残りから、SuperGPQA は残りの問題から、MATH は train 分割の Level 3~5 から抽出した。世代  $t$  の議論には  $Q_{t-1}$  を用い、同じ問題を二度学習に使わない。正解は、多肢選択では正解の選択肢の記号、MATH では解答例の最後の `\boxed{}` の中身（MATH-500 では収録された正解）とした。

## 4.3 採点

### 4.3.1 回答の抽出と判定

正誤の判定は、生成文から回答を取り出す抽出と、取り出した回答を正解と照合する判定の 2 段からなる（2.6 節）。システムプロンプトは、最終行に `ANSWER: <letter>`（MATH では `ANSWER: <final simplified answer>`）の書式で答えるよう指示する。

**抽出。** 生成文の最後の `ANSWER:` 行を探し、同じ行にある文字列を捕捉する。同じ行が空なら、次の空でない行を用いる。多肢選択では、捕捉した文字列から次の順で選択肢の記号を取り出す。

1. 先頭にある記号（H, `**H**`, (H), `\boxed{H}` など）。
2. 「Option C」「answer is C」のようなキーワードに続く記号。

3. 独立した1文字のトークンとしての記号. 複数ある場合は, 一人称の「I」を除いた最後の記号.

ANSWER: 行がない場合は, 本文全体から「the answer is X」「 $\boxed{X}$ 」「**\*\*X\*\***」の形の最後の一致を用いる. MATH では, ANSWER: 行の内容 (その中に  $\boxed{\}$  があればその中身) を取り出し, 行がなければ本文の最後の  $\boxed{\}$  の中身を用いる.

**判定.** 多肢選択は記号の一致で判定する. MATH は, まず表記を正規化して比較する. 正規化では,  $\text{\texttt{\text{text}}}$  や  $\text{\texttt{\text{left}}}$ , 空白, 度やパーセントの記号, 先頭の「 $x =$ 」を除き, 分数や根号を統一した形に直し, 数値は数として比べる. 一致しなければ, Math-Verify[117] (0.8.0) による記号的な同値判定を用いる (1問あたり5秒で打ち切る). test の MATH 300 問の正解は, すべて Math-Verify で解析できることを確認した.

回答を抽出できなかった出力は誤答として扱う. 抽出できない出力の割合 (None 率) は条件ごとに第5章で報告し, 条件間で大きく異ならないことを確かめる.

#### 4.3.2 予備研究の採点器の監査

本研究の採点器は, 予備研究の採点器に見つかった次の3つの欠陥を直したものである.

1. **改行をまたぐ捕捉:** 旧実装は, ANSWER: の後の空白に改行を含めて捕捉していた. そのため, 「Final Answer:」の次の行に「ANSWER: H」と書いた出力から, 単語 ANSWER の頭文字の「A」を回答として取り出していた. 予備研究で保存したベースモデルの10択の呼び出し34,608件に新旧の抽出器を適用すると, 9.1% (3,142件) で結果が変わり, すべて旧が「A」だった.
2. **最初の文字の採用:** 旧実装は, 捕捉した文字列に最初に現れる A~J の文字を回答としていた. そのため「ANSWER: Option C」を「I」と判定していた.
3. **数式の偽陰性:** 旧実装は, 簡単な文字列の正規化だけで数式を比べていた. そのため  $\sqrt{2}$  と  $\sqrt{\{2\}}$ ,  $\frac{59}{\}$  と  $\frac{\{5\}}{\{9\}}$ , 度の記号や「 $x =$ 」の有無といった表記の違いを誤答としていた. 予備研究の MATH の予測9,600件のうち, 6.49% (623件) が偽陰性だった.

これらの欠陥は条件によって偏って効いていた。1 番目の欠陥で結果が変わった呼び出しの割合は、ベースモデルと SC では 10.6~16.8% だったのに対し、出力の書式を SFT で揃えた LoRA のチームでは 0.0~0.1% だった。採点器の誤りが条件によって偏れば、条件間の比較そのものが歪む。そこで本研究では、採点器を評価の前に固定し、保存した生出力に新旧の抽出器を適用して条件ごとに誤りを監査した。予備研究の結果を修正した採点器で採点し直した結果は、第 5 章で報告する。

## 4.4 系統と比較条件

### 4.4.1 進化の系統

進化の系統を表 4.2 に示す。全系統で、世代 0 の社会（3.2 節で採用したトリオ）、議論のプロトコル、ゲーティング、学習用の問題、学習の設定を共通にする。系統の違いは選抜の規則と、それに伴う候補の作り方だけである。

表 4.2: 進化の系統

| 系統     | 選抜の規則                             | 候補                   | 世代数          | test の評価 | 目的           |
|--------|-----------------------------------|----------------------|--------------|----------|--------------|
| S (提案) | 代表チーム文脈の厳密 Shapley 値 (dev, macro) | 親, a, b, c           | 3 (予算が許せば 4) | 各世代      | 提案手法         |
| N      | なし (常に子 b)                        | b (世代 2 以降は b だけを作る) | 3            | 最終世代     | 選抜の効果 (RQ2)  |
| A1     | 単独の精度 (dev, macro)                | 親, a, b, c           | 3            | 最終世代     | 適応度の水準 (RQ3) |

世代 1 の子 (a, b, c) は全系統で共有する。同じ親から同じデータで学習するため、系統ごとに作り直す必要がないからである。系統が分かれるのは世代 1 の選抜からであり、世代 2 以降は各系統が自分の代表を親として子を作る。系統 N は選抜を行わないため、世代 2 以降は子 b だけを学習し、子 a の学習と交叉は行わない。

系統 S は各世代の社会を test で評価し、世代ごとの推移を記録する。この評価は選抜には用いない。系統 N と A1 は、費用を抑えるため最終世代だけを評価する。

#### 4.4.2 単一モデルの自己学習 (RFT)

「社会の進化は、同じ問題で単に自己学習した場合に比べて何を加えるのか」を測る対照として、ペルソナも議論も用いない単一モデルの棄却サンプリング微調整 (rejection sampling fine-tuning; RFT) [19, 20] を置く。

ベースモデル (ペルソナ文なし) で、系統 S が 3 世代で用いる学習用の問題 ( $Q_0 \sim Q_2$  の計 3,600 問) を 1 問あたり 2 回サンプリングする。生成が完了した正解の出力を 1 問につき 1 例まで集め、上限を 3,000 例とする。この上限は、1 体のペルソナが 3 世代の学習で使う例数の上限 (1,000 例  $\times$  3 世代) に等しい。これで新しい LoRA を 1 エポック学習する (学習率  $1 \times 10^{-4}$ , 他の設定は系統と同じ)。学習したモデルで test の各問題を 9 回サンプリングし、単独の精度と  $SC@k$  を求める。

#### 4.4.3 基準

- **ベースモデル単体**: ペルソナ文なしで test の各問題を 9 回サンプリング (生成 seed 1) し、その平均正答率を単独の精度とする。これは 1 回の生成の期待精度である。
- **$SC@k$  ( $k = 1, \dots, 9$ )** [7]: 同じ 9 本の標本のうち最初の  $k$  本の多数決。同数の処理は 3.3.4 項と同じである。副次的な  $SC@k$  の曲線 (4.5.3 項) では、9 本から  $k$  本を選ぶ全組合せでの正答率の平均 (期待値) を用い、標本の順序による揺らぎを除く。
- **世代 0 の社会**: 採用したトリオ (LoRA なし) を、系統と同じプロトコルとゲーティングで評価する。主要比較 H1 (4.5 節) の基準である。
- **7 月のチーム**: 予備研究で最良だったチーム (ベースモデル自身の正解を混ぜた SFT で学習した LoRA と、日本語のペルソナ文) を、本研究の test で評価する。議論は予備研究の手続き (自分の前回の回答を提示せず、匿名化した他者の回答と条件付きの指示を一つの user の発話で与える) を本環境で再実装したもので、ゲーティングは行わない。集約は本研究の規則 (3.3.4 項) に揃える。予備研究の手法の位置を、修正した採点器と新しい test で確かめるための条件である。

#### 4.4.4 生成の回数と出力トークン数の考慮

議論のチームは、1問あたり round0 で3回、round1 で最大3回、合わせて最大6回の生成を行い、ゲーティングの下ではそれより少ない。議論はこれと同程度の生成回数の Self-Consistency と比べるべきだという批判がある [8]。そこで、最終世代の社会と SC@ $k$  を、1問あたりの生成回数と出力トークン数をそろえた点で比べる (RQ4)。round1 は他者の回答を入力に含むため入力トークンが多く、この点は SC に有利な比較となる。入力トークン数も併せて報告する。

### 4.5 評価指標と統計

#### 4.5.1 評価指標

主要な評価指標は、test のベンチマーク等重み (macro) の精度

$$\text{Acc}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K \frac{1}{n_k} \sum_{i \in I_k} s_i \quad (4.1)$$

である。ここで  $I_k$  はベンチマーク  $k$  の問題の集合、 $n_k = |I_k|$ 、 $s_i \in [0, 1]$  は問題  $i$  の得点 (生成 seed が複数あればその平均正答) である。副次的な指標として、問題数で重み付けした精度 (micro) とベンチマーク別の精度を報告する。

#### 4.5.2 主要な比較と検定

主要な比較は次の3つであり、結果を見る前に定めた。

- **H1** (RQ1, 進化の効果) : 系統 S の最終世代の社会の精度は、世代 0 の社会より高い。
- **H2** (RQ2, 選抜の効果) : 系統 S の最終世代の精度は、系統 N の最終世代より高い。
- **H3** (RQ3, 適応度の水準) : 系統 S の最終世代の精度は、系統 A1 の最終世代より高い。

3つの検定の族について，Holmの方法[109]で族全体の第1種の誤りの確率を5%に抑える．

検定は，問題を単位とする対応のある比較である[107]．2条件の問題ごとの得点の差を  $d_i$  とし，検定統計量をベンチマーク内の平均差の等重み平均

$$\Delta = \frac{1}{K} \sum_{k=1}^K \frac{1}{n_k} \sum_{i \in I_k} d_i \quad (4.2)$$

とする． $p$  値は，問題ごとに差の符号を無作為に反転させる符号反転の並べ替え検定（両側，20,000回）で求める．反転後の統計量  $\Delta^{(b)}$  から， $p = (1 + \#\{b : |\Delta^{(b)}| \geq |\Delta|\}) / (B + 1)$  とする（ $B = 20,000$ ）．信頼区間は，ベンチマーク内で問題を復元抽出するブートストラップ（10,000回）のパーセンタイル区間とする．生成 seed が複数ある場合は，問題ごとに平均してから比べるので，問題が検定の単位（クラスタ）になる．比較には，両条件の結果がそろった問題だけを用いる．

主要比較のチーム（世代0と各系統の最終世代）は，test を生成 seed 2本で評価し，問題ごとに2本の平均を得点とする．途中の世代は生成 seed 1本で評価する．事前登録からの逸脱があれば，その理由を第5章で報告する．同等性を主張する場合は，事前に定めたマージン  $\pm 2pt$  を用い，差の90%信頼区間がこの範囲に収まるときに限る．

### 4.5.3 副次的な分析

次の分析は探索的なものであり，多重性の補正は行わず，信頼区間とともに報告する．

- 系統  $S$  の世代ごとの精度の推移．
- $SC@k$  の曲線に対する各手法の位置（生成回数と出力トークン数をそろえた比較）．
- 議論における正誤の遷移（round0で少数派の正解が多数派に合わせた割合，多数派が答えを維持した割合など）．
- ペルソナ間の回答の多様性（不一致率と，3体のうち少なくとも1体が正解する割合  $oracle@3$ ）の世代ごとの推移．
- 候補の Shapley 値と単独の精度の順位相関，および Shapley 値と Banzhaf 値の順位の一致．

## 4.6 ハイパーパラメータと設計値

### 4.6.1 ハイパーパラメータ

生成と推論の設定を表 4.3 に、議論と進化の設定を表 4.4 に、LoRA の学習の設定を表 4.5 に示す。

表 4.3: 生成と推論の設定

| 項目                       | 値                                  |
|--------------------------|------------------------------------|
| 温度 / top-p / top-k       | 0.7 / 0.8 / 20                     |
| 出力の上限                    | 8,192 トークン（全条件・全ラウンド共通）            |
| コンテキスト長の上限               | 49,152 トークン                        |
| 同時に処理する系列の上限             | 256                                |
| GPU メモリの使用率の上限           | 0.90                               |
| LoRA の同時搭載数の上限 / 最大 rank | 12 / 16                            |
| クライアントの並列数               | 192                                |
| 完全反復の打ち切り                | 周期 5 回以上かつ 1,500 文字以上（400 文字ごとに検査） |
| 確信度                      | 発話の平均トークン対数確率                      |

### 4.6.2 パイロットで決めた設計値

本実験の前に、dev だけを用いるパイロットを 2 回行い、議論のプロトコル、ゲーティング、クライアントの並列数を決めた。test は用いていない。

**議論のプロトコル.** 第 1 パイロットで、切り詰めない全文版の round1 は、入力平均で約 7,600 トークンになることが分かった。このとき、A100 40GB の KV キャッシュには同時に約 20 本の系列しか載らず、出力の速度は毎秒約 500 トークンに落ちた。この速度では、1 世代の選抜に必要な約 2.5 万回の round1 が予算に収まらない。そこで第 2 パイロットの前に、round1 の採用候補を切り詰め版の 2 種に改めた（3.3.2 項）。dev-A での既定トリオの macro 精度を表 4.6 に示す。規則どおり、中立版の切り詰め版を採用した。全文版に比べて、切り詰めによる精度の低下は見られなかった。なお、dev-A は 200 問であり、これらの差の大きさは標本誤差と同程度である。

**ゲーティング.** 第 1 パイロットでは、round0 で多数派だったエージェントが round1 で答えを維持した割合は 93~95% だった。dev-A の既定トリオでは、ゲーティング

表 4.4: 議論と進化の設定

| 項目               | 値                                              |
|------------------|------------------------------------------------|
| 役割の数             | 3 (世代 0 で選定, 全世代で固定)                           |
| ラウンド             | round0 (独立回答) と round1 (更新) の 1 回              |
| round1 の指示       | 中立版 (3.3.1 項)                                  |
| round1 での回答の切り詰め | 自分と他者の回答をそれぞれ末尾 3,000 文字                       |
| ゲーティング           | あり (進化の各系統. 世代 0 の選定と 7 月のチームはなし)              |
| 集約               | 多数決 → 確信度の最大値 → 役割名 → 回答の文字列                   |
| 選抜に用いる問題         | dev 全体 (400 問, macro 精度)                       |
| 1 世代の学習用の問題      | 1,200 問 (世代ごとに新しい組)                            |
| 世代数 $G$          | S: 3 (予算が許せば 4), N・A1: 3                       |
| 役割ごとの候補          | 親, 子 a, 子 b, 子 c                               |
| 学習例の上限           | 子 1 体あたり 1,000 例                               |
| 兄弟交叉             | $\alpha = 0.5$ , rank 16 に再分解                  |
| ランダム化 SVD        | 過剰標本 8, べき乗反復 4 回                              |
| 生成 seed          | 議論・選抜・途中世代の test は 1. 世代 0 と最終世代の test は 1 と 2 |

表 4.5: LoRA の学習の設定

| 項目                                      | 値                                                     |
|-----------------------------------------|-------------------------------------------------------|
| ベースモデルの精度                               | bf16 (量子化なし)                                          |
| rank / $\alpha_{\text{LoRA}}$ / dropout | 16 / 32 / 0.05                                        |
| 対象モジュール                                 | $q$ , $k$ , $v$ , $o$ , gate, up, down 射影             |
| 損失を掛ける範囲                                | 最後の assistant の発話と終端トークン                              |
| 1 回の更新あたりのラベルトークン                       | 32,768                                                |
| エポック数                                   | 1                                                     |
| 学習率                                     | 新しい LoRA $1 \times 10^{-4}$ , 継続学習 $5 \times 10^{-5}$ |
| 学習率の予定                                  | 5% のウォームアップの後, 余弦減衰で 0 へ                              |
| 最適化                                     | AdamW ( $\beta = (0.9, 0.999)$ , 重み減衰 0)              |
| 勾配のクリッピング                               | ノルム 1.0                                               |
| 例の最大長                                   | 8,192 トークン (超える例は除く)                                  |
| seed                                    | 個体名のハッシュから決定的に定める                                     |

表 4.6: 議論のプロトコルの比較 (dev-A, 既定トリオ, ゲーティングなし, macro 精度)

| プロトコル               | 自分の回答の提示         | 指示   | macro 精度     |
|---------------------|------------------|------|--------------|
| 採用候補 (切り詰め版)        |                  |      |              |
| <b>切り詰め・中立 (採用)</b> | あり (末尾 3,000 文字) | 中立   | <b>0.784</b> |
| 切り詰め・条件付き           | あり (末尾 3,000 文字) | 条件付き | 0.780        |
| 参照 (全文版)            |                  |      |              |
| 全文・中立               | あり (全文)          | 中立   | 0.764        |
| 全文・条件付き             | あり (全文)          | 条件付き | 0.791        |
| 7 月の手続き             | なし               | 条件付き | 0.782        |

なしの精度 0.784 に対し, ゲーティングありでは 0.789 であり, 精度の低下は見られなかった. round0 で 3 体の回答が全員一致した問題は 59% あり, その分の round1 を省ける. この結果に基づき, 本実験の進化と評価にゲーティングを導入した.

**クライアントの並列数.** 独立な生成の出力の速度は, 並列数 128 以上でほぼ頭打ちになった (4.7.1 項). 本実験は並列数 192 で行う.

**完全反復の打ち切り.** パイロットでは, 生成の数%が出力の上限 8,192 トークンに達したが, いずれも完全な反復ではなく内容の変わる迷走であり, 打ち切りは一度も発動しなかった. 正答した出力を誤って打ち切る例もなかったため, 打ち切りは有効のまま用いる.

### 4.6.3 事前登録からの変更

設計の文書を固定した後に行った変更を, 日時と理由とともに次に示す. いずれも dev での結果と計算量の実測だけにに基づいて行い, test の結果は判断に用いていない.

1. **コンテキスト長の上限の拡張** (2026 年 9 月 30 日): 全員が迷走した問題では, round1 の入力 that 約 2.5 万トークンになり, 出力の上限 8,192 トークンと合わせて vLLM の上限 32,768 トークンを超えた. 上限を 49,152 トークンに拡張した. 生成の内容は変わらない.
2. **出力の上限の維持** (同日): 上限 8,192 トークンに達した出力は完全な反復ではなかった. 途中で切ると条件間で扱いが変わりうるため, 上限は全条件で共通のまま維持し, 完全反復の打ち切りだけを残した.

3. **round1 の採用候補の変更**（同日，第 2 パイロットの前）：計算量の制約から，採用候補を切り詰め版の 2 種に改め，全文版は参照とした（4.6.2 項）。
4. **ゲーティングの導入**（同日，本実験の前）：dev-A での比較に基づき，進化と評価に導入した（4.6.2 項）。
5. **対照系統の評価の縮小**（同日，本実験の前）：費用を抑えるため，系統 N と A1 の test の評価を最終世代だけとした。主要比較 H2 と H3 は最終世代同士の比較なので，この変更の影響を受けない。
6. **社会学習のデータの構成**（同日，本実験の前）：当初は，子 b のデータを「a の例と他者の正解例を合わせた中から 1,000 例を無作為に抽出」としていた。しかし 1,200 問の議論からは自己学習の例が約 2,000 例得られるのに対し，他者の例は数百例にとどまる。無作為に抽出すると他者の例の約半分が落ち，b が a とほとんど同じになる。a と b の違いを社会学習の成分として保つため，他者の例をすべて含め，残りを自己学習の例で補う構成に改めた（3.5.2 項）。

## 4.7 計算資源と予算

### 4.7.1 計算資源

計算は Google Cloud の Vertex AI のカスタムジョブで行い，1 ジョブに NVIDIA A100 (40GB) を 1 枚用いる。GPU は Spot VM（中断されうる安価な枠）か，Flex-start（起動を待つ代わりに，起動後は中断されない枠）で確保する。Spot VM の在庫不足に備え，同じジョブを複数のリージョン（us-central1, europe-west4, asia-southeast1）に投入し，最初に起動したものだけを続けて残りを取り消す。

ジョブは中断と再開を前提に設計した。すべての生成は一意のキーでクラウドストレージに保存し，学習した個体も個体ごとに保存する。ジョブが中断して再開したときは，保存済みの生成と工程を飛ばして続きから実行する。

パイロットで測った出力の速度（概数）は，独立な生成（round0 と SC）で毎秒約 2,100 トークン（並列数 64 では約 2 割低い），全文版の round1 で毎秒約 500 トークン，切り詰め版の round1 で毎秒約 1,100 トークンだった。round1 が遅いのは，入力が長く，KV キャッシュに同時に載る系列が少ないためである。

### 4.7.2 予算と中止規則

計算の費用は、クラウドの無料クレジット ¥47,813 (300 米ドル) で賄う。これは超えられない上限であり、研究はこの範囲で完結させる。実際の費用は課金データのエクスポートで確認する。エクスポートに反映されるまでの間は、ジョブの稼働時間に実効単価 (1 時間あたり 2.2 米ドル) を掛けて見積もる。この単価は、予備研究で測った Vertex AI の GPU ジョブの実効単価 (Spot VM の単価の約 2 倍) に基づく。

事前に定めた費用の配分の目安を表 4.7 に示す。中止規則は次のとおりである。

- 累計の費用が ¥36,000 (上限の 75%) に達したら、新しい系統を始めない。
- ¥42,000 (88%) に達したら、すべてのジョブを止め、手元のデータで論文にまとめる。
- 世代 1 の dev で、子 (a, b, c) がすべて親より 2pt 以上悪い場合は、学習の設定を見直す。見直すのは学習率と例数だけとし、設計は変えない。

予算のアラートは ¥45,000 に設定した。パイロットで測った処理速度に基づく見積もりの見直しと、実際の費用は第 5 章で報告する。

表 4.7: 費用の配分の目安 (事前登録時。予備を含めて上限 300 米ドルの範囲に収める)

| 項目              | 金額 (米ドル) |
|-----------------|----------|
| パイロット           | 5        |
| 基準の評価           | 25       |
| 系統 S            | 45       |
| 系統 N と A1       | 25       |
| RFT             | 10       |
| 最終評価の追加の生成 seed | 30       |
| 予備              | 100 以上   |

## 第5章 実験結果

本章では、5.1 節で予備研究の再検証の結果を、5.2 節で世代 0 のペルソナの選定の結果を、5.3 節で進化的社会学習の結果 (RQ1~RQ3) を、5.4 節で基準との比較 (RQ4) を、5.5 節で議論と選抜の分析を報告する。特に断らない限り、精度はベンチマーク等重み (macro) の正答率、差は pt (百分率の差)、括弧内は 95% 信頼区間である。

### 5.1 予備研究の再検証

#### 5.1.1 予備研究の構成

予備研究 (2026 年 7 月) では、日本語のペルソナ文を持つ 3 体 (批判的検証者、実務的意思決定者、発散的探索者) の LoRA を、それぞれ 60 件の対話データで学習させた (以下、旧チーム)。議論では、各エージェントが独立に回答した後、他の 2 体の回答を読んで回答を更新し、多数決で最終回答を決めた。この旧チームを初期集団として、代表チーム文脈の厳密 Shapley 値に fitness sharing を掛けた適応度で、LoRA の重みを交叉と突然変異により 6 世代進化させた。適応度は MMLU-Pro の 100 問で測った。その後、能力の低下を補うため、ベースモデル自身の正解の長い推論を混ぜた対話データで 3 体を学習し直し、議論の手続きに条件付きの更新、発言の匿名化、確信度で重み付けした投票を加えたチームを作った (以下、7 月のチーム)。評価は MMLU-Pro 500 問、MATH-500 の Level 4-5 の 200 問、SuperGPQA 300 問を 3~6 本の生成 seed で行った。

#### 5.1.2 修正した採点器による再採点

予備研究で保存したすべての生出力を、本研究の採点器 (4.3 節) で採点し直した。結果を表 5.1 に示す。ベースモデルの単独の精度は、1 問あたり 9 本の標本の平均正答率である。

表 5.1: 予備研究の結果の再採点（修正した採点器，予備研究の問題集合）

| 条件              | MMLU-Pro | MATH  | SuperGPQA | macro |
|-----------------|----------|-------|-----------|-------|
| ベースモデル単体（9本の平均） | 0.723    | 0.903 | 0.445     | 0.691 |
| SC@3            | 0.732    | 0.930 | 0.469     | 0.710 |
| SC@6            | 0.737    | 0.941 | 0.479     | 0.719 |
| SC@9            | 0.741    | 0.939 | 0.484     | 0.721 |
| 7月のチーム          | 0.716    | 0.933 | 0.429     | 0.693 |

7月のチームはSC@9に $-2.86\text{pt}$  ( $[-3.73, -1.98]$ ,  $p < 10^{-4}$ ), SC@3にも $-1.76\text{pt}$  ( $[-2.64, -0.88]$ ,  $p = 0.0001$ ) 及ばなかった．ベースモデル単体（9本の平均）との差は $+0.22\text{pt}$  ( $[-0.58, +1.02]$ ,  $p = 0.60$ ) で，差は検出されなかった．ベンチマーク別に見ても，チームがSC@9を上回ったベンチマークはなかった（MMLU-Pro  $-2.5\text{pt}$ , MATH  $-0.6\text{pt}$ , SuperGPQA  $-5.5\text{pt}$ )．チームが3～6回の生成を使うのに対し，同じ回数以下の生成の多数決にも及ばなかったことになる．

予備研究の論文の草稿では、「議論の効果がベンチマークによって反転する（MMLU-Pro でだけチームが有効で，素の議論は有害）」と解釈していた．しかし再検証の結果，この反転はMMLU-Proのベースモデル系の条件だけが採点器の欠陥（4.3.2項）で過小評価されていたことによるもので，実在しなかった．同様に「評価環境を作り直したことで精度が $+6\text{pt}$ 変わった」とした観察も，環境の違いではなく，途中で修正した採点器が一部の結果にだけ適用されていたことによるものだった．

### 5.1.3 予備研究が失敗した理由

保存済みの生出力と進化の記録を解析し，予備研究のチームと進化が機能しなかった理由を特定した．

**ペルソナの学習による能力の低下．** 学習に用いた対話データの応答は平均779文字と短く，学習後のエージェントは推論の記述を約半分（ベースモデルの平均2,423文字に対し1,188文字）に縮めた．その結果，長い導出を要するMATHのLevel 5の問題で精度が大きく下がった．進化後の旧チームは，ベースモデル単体に $-2.62\text{pt}$  ( $[-4.31, -0.91]$ ) 及ばなかった．

**多様性の欠如．** 同じ問題集合で，異なるペルソナのエージェント同士の回答の不一致率と，同じエージェントを乱数だけ変えて再サンプリングしたときの不一致率を比べると，差は $-0.9 \sim +1.4$ ポイントにすぎなかった．3体のうち少なくとも1体が正解する割合（oracle@3）も，ベースモデルの3本の標本のoracle@3を

上回らなかった。ペルソナの学習は、再サンプリング以上の多様性を生んでいなかった。

**同調.** 予備研究の議論は、更新のラウンドでエージェントに自分の前回の回答を提示していなかった。このため round0 で少数派だったエージェントは、自分が正解であっても 81~95% の確率で多数派の誤答に答えを変えた。少数派の正解が議論で生き残る経路がほぼなかった。

**選抜のノイズ.** 進化の記録では、同じ連合を同じ条件で測り直した値が単独で標準偏差 3.2pt 揺れた。この揺れを伝播させると、候補間の Shapley 値の差のノイズの標準偏差は約 1.9pt になる。一方、実際に観測された候補間の差の中央値は 1.2pt であり、18 回の選抜はすべてノイズの 2 標準偏差以内の差で決まっていた。観測された差の分散は純粋なノイズの期待値を下回っており、候補間に真の差の分散は検出されなかった。選抜時の最大値は次の世代の再測定で平均 3.2pt 下がり、開発用の問題での「+12pt」の改善は、毎世代の最大値を拾う勝者の呪いとして説明できた。加えて、交叉と突然変異は重みをほとんど動かしておらず、進化の前後の  $\Delta W$  のコサイン類似度は全モジュールで 0.9996 以上だった。候補間に差がなく、差があっても測れない状態だったことになる。予備研究のデータを再標準化して試算すると、真の差が約 3pt の 2 条件で正しい方を選ぶ確率は、無作為に選んだ 100 問では 0.77~0.78 にとどまるが、正答率が中間の問題を 70% に濃縮した 300 問では 0.96~0.97 になる。

#### 5.1.4 本研究への示唆

以上の再検証から、本研究の設計を次のように定めた（第 3 章，第 4 章）。

- ペルソナは性格のラベルではなく推論の手続きとして定義し、ペルソナ文だけで与える（学習による能力の低下を避ける）。候補は理論から網羅的に作り、チームとしての成績で選ぶ。
- 議論では自分の前回の回答を会話履歴に保持する（同調の実装上の原因を除く）。
- 候補間の差は、重みの乱数的な摂動ではなく、議論の正解発話からの学習で作る。

- 選抜には 400 問の dev を用い，選抜の単位ごとに全 7 連合を実測する．最終評価の test は選抜と分ける．
- 採点器は監査して固定し，すべての結果を同じ採点器で採点する．

## 5.2 世代 0 のペルソナの選定

## 5.3 進化的社会学習

## 5.4 基準との比較

## 5.5 議論と選抜の分析

## 第6章 考察

### 6.1 本研究の限界

本研究の結論には次の限界がある。

**システムの反復がない。** 各システム (S, N, A1) と RFT は、予算の制約からそれぞれ 1 回ずつしか実行していない。4.5 節の検定は、問題の標本抽出と生成の揺らぎを考慮するが、学習の乱数 (例の抽出順や初期値) や選抜の偶然による系統ごとのばらつきは含まない。したがって系統間の差は「この 1 回の実行での差」であり、同じ手続きを繰り返したときの平均的な差を直接には示さない。

**選抜の精度。** 選抜に用いた dev は、正答率が中間の問題を 70% に濃縮した 400 問である。予備研究のデータによる試算では、真の差が約 3pt の候補の組で正しい方を選ぶ確率は、同じ濃縮設計の 300 問で 0.96~0.97 である (5.1.3 項)。一方、差が 1~2pt の候補の選別には、数千問規模の評価が必要になる。本研究の選抜は、大きな差のある候補を見分けることはできても、僅差の候補の順位は偶然に左右されうる。

**規模と範囲。** ベースモデルは 1 種類 (Qwen3-4B-Instruct-2507)、役割は 3 体、世代数は 3 に限られる。より大きなモデル、異なるモデルの混成、より多くの役割や世代での振る舞いは検証していない。ベンチマークは多肢選択の 2 種と数学の 1 種であり、自由記述の推論や対話の課題は含まない。

**設計上の選択。** 議論は 1 ラウンドとし、round1 の入力は末尾 3,000 文字に切り詰め、全員一致の問題では議論を省いた。出力は 8,192 トークンで打ち切った。これらは計算量の制約から選んだものであり、パイロットでは精度を損なわないことを確かめたが、条件を変えれば結果が変わる可能性はある。ペルソナの候補は、Belbin の役割を著者が推論の手続きとして操作化したものであり、別の操作化もありうる。

**採点と再現性。** 採点器は監査して固定したが、数学の同値判定には規則ベースの検証器の誤りが残りうる。vLLM の生成は同じ seed でも再現しないため、本研

究は生成を一度だけ行って保存し，分析をその保存に対して再現可能にした．生成そのものの再実行で同じ数値が得られることは保証しない．

## 第7章 結論

# 謝辞

本研究が順調に展開できたのは、指導教員の余俊助教から提供いただいた実験環境と貴重な指導、また、研究室全員からはいろいろな貴重なご意見をいただいた。心から感謝します。

# 参考文献

- [1] Junyou Li et al. More agents is all you need. *Transactions on Machine Learning Research*, 2024. arXiv:2402.05120.
- [2] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2305.14325.
- [3] Tian Liang et al. Encouraging divergent thinking in large language models through multi-agent debate. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024. arXiv:2305.19118.
- [4] Edward J. Hu et al. LoRA: Low-rank adaptation of large language models. arXiv:2106.09685 (ICLR 2022), 2021.
- [5] Qwen Team. Qwen3 technical report. arXiv:2505.09388, 2025.
- [6] Robert Boyd and Peter J. Richerson. *Culture and the Evolutionary Process*. University of Chicago Press, 1985.
- [7] Xuezhong Wang et al. Self-consistency improves chain of thought reasoning in language models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023. arXiv:2203.11171.
- [8] Andries Smit, Paul Duckworth, Nathan Grinsztajn, Thomas D. Barrett, and Arnu Pretorius. Should we be going MAD? A look at multi-agent debate strategies for LLMs. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2311.17371.
- [9] Hangfan Zhang, Zhiyao Cui, Jianhao Chen, Xinrun Wang, Qiaosheng Zhang, Zhen Wang, Dinghao Wu, and Shuyue Hu. Stop overvaluing multi-agent debate — we must rethink evaluation and embrace model heterogeneity. arXiv:2502.08788, 2025.
- [10] Hyeon Kyu Choi, Xiaojin Zhu, and Sharon Li. Debate or vote: Which yields better decisions in multi-agent large language models? In *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025. arXiv:2508.17536, NeurIPS 2025 Spotlight.
- [11] Iliya Mirzaei. Sample more, reflect less: Self-refine and reflexion lose to repeated sampling at equal token cost, from 1.5B to 7B. arXiv:2607.28576, 2026.
- [12] Blaž Bertalančič and Carolina Fortuna. The cost of consensus: Isolated self-correction prevails over unguided homogeneous multi-agent debate. arXiv:2605.00914, 2026. ACM Conference on AI and Agentic Systems.

- [13] Mingqian Zheng, Jiaxin Pei, Lajanugen Logeswaran, Moontae Lee, and David Jurgens. When “a helpful assistant” is not really helpful: Personas in system prompts do not improve performances of large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024. arXiv:2311.10054.
- [14] Junseok Kim, Nakyeong Yang, and Kyomin Jung. Persona is a double-edged sword: Mitigating the negative impact of role-playing prompts in zero-shot reasoning tasks. arXiv:2408.08631, 2024.
- [15] Yexiang Liu, Jie Cao, Zekun Li, Ran He, and Tieniu Tan. Breaking mental set to improve reasoning through diverse multi-agent debate. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.
- [16] R. Meredith Belbin. *Management Teams: Why They Succeed or Fail*. Heinemann, 1981.
- [17] R. Meredith Belbin. *Team Roles at Work*. Butterworth-Heinemann, 1993.
- [18] Darrell Whitley, V. Scott Gordon, and Keith Mathias. Lamarckian evolution, the Baldwin effect and function optimization. In *Parallel Problem Solving from Nature (PPSN III)*, volume 866 of *Lecture Notes in Computer Science*, pages 5–15. Springer, 1994. DOI: 10.1007/3-540-58484-6.245.
- [19] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022. arXiv:2203.14465.
- [20] Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. Scaling relationship on learning mathematical reasoning with large language models. arXiv:2308.01825, 2023.
- [21] Lars Benedikt Kaesberg, Jonas Becker, Jan Philip Wahle, Terry Ruas, and Bela Gipp. Voting or consensus? Decision-Making in multi-agent debate. In *Findings of the Association for Computational Linguistics: ACL 2025*, 2025. arXiv:2502.19130.
- [22] Quim Motger, Marc Oriol, Jordi Marco, and Xavier Franch. Multi-agent debate strategies: Survey, taxonomy, and challenges. arXiv:2607.26212, 2026.
- [23] Andrea Wynn, Harsh Satija, and Gillian Hadfield. Talk isn’t always cheap: Understanding failure modes in multi-agent debate. arXiv:2509.05396, 2025. ICML 2025 MAS Workshop.
- [24] Jintian Zhang, Xin Xu, Ningyu Zhang, Ruibo Liu, Bryan Hooi, and Shumin Deng. Exploring collaboration mechanisms for LLM agents: A social psychology view. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2310.02124.
- [25] Haolun Wu, Zhenkun Li, and Lingyao Li. Can LLM agents really debate? A controlled study of multi-agent debate in logical reasoning. arXiv:2511.07784, 2025.
- [26] Maya Okawa. Emergence of biased consensus in multi-agent LLM debates. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2608.02827.
- [27] Aneesh Pappu, Batu El, Hancheng Cao, Carmelo di Nolfo, Yanchao Sun, Meng Cao, and James Zou. Multi-agent teams hold experts back. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.01011.

- [28] Justin Chih-Yao Chen, Swarnadeep Saha, and Mohit Bansal. ReConcile: Round-table conference improves reasoning via consensus among diverse LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2309.13007.
- [29] Xiaochen Zhu, Caiqi Zhang, Yizhou Chi, Tom Stafford, Nigel Collier, and Andreas Vlachos. Demystifying multi-agent debate: The role of confidence and diversity. arXiv:2601.19921, 2026.
- [30] Wenzhe Li, Yong Lin, Mengzhou Xia, and Chi Jin. Rethinking mixture-of-agents: Is mixing different large language models beneficial? arXiv:2502.00674, 2025.
- [31] Shubham Toshniwal, Ivan Sorokin, Aleksander Ficek, Ivan Moshkov, and Igor Gitman. GenSelect: A generative approach to best-of-N. arXiv:2507.17797, 2025. 2nd AI for Math Workshop at ICML 2025.
- [32] Siddarth Venkatraman, Vineet Jain, Sarthak Mittal, Vedant Shah, Johan Obando-Ceron, Yoshua Bengio, Brian R. Bartoldson, Bhavya Kailkhura, Guillaume Lajoie, Glen Berseth, Nikolay Malkin, and Moksh Jain. Recursive self-aggregation unlocks deep thinking in large language models. arXiv:2509.26626, 2025.
- [33] Guiv Farmanfarmanian. Selection, recombination, or a fresh solve? A candidate-free control for single-pass test-time aggregation. arXiv:2608.18379, 2026. COLM 2026 Workshop on Efficient Reasoning.
- [34] Carolina Fortuna and Blaž Bertalanč. Multi-agent scaling across disjunctive and compensatory tasks. arXiv:2609.31563, 2026.
- [35] Zizhao Hu, Mohammad Rostami, and Jesse Thomason. Expert personas improve LLM alignment but damage accuracy: Bootstrapping intent-based persona routing with PRISM. arXiv:2603.18507, 2026.
- [36] Zhenhailong Wang, Shaoguang Mao, Wenshan Wu, Tao Ge, Furu Wei, and Heng Ji. Unleashing the emergent cognitive synergy in large language models: A task-solving agent through multi-persona self-collaboration. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024. arXiv:2307.05300.
- [37] Aryan Keluskar, Amrita Bhattacharjee, and Huan Liu. When does personality composition matter for multi-agent LLM teams? In *Proceedings of the Conference on Language Modeling (COLM)*, 2026. arXiv:2606.27443.
- [38] Luke Baines et al. Persona cartography: Charting language model personality traits in weight space. arXiv:2607.07916, 2026.
- [39] Adrian Furnham, Howard Steele, and David Pendleton. A psychometric assessment of the Belbin team-role self-perception inventory. *Journal of Occupational and Organizational Psychology*, 66(3):245–257, 1993. DOI: 10.1111/j.2044-8325.1993.tb00535.x.
- [40] Barbara Senior. Team roles and team performance: Is there ‘really’ a link? *Journal of Occupational and Organizational Psychology*, 70(3):241–258, 1997. DOI: 10.1111/j.2044-8325.1997.tb00646.x.

- [41] Aitor Aritzeta, Stephen Swailes, and Barbara Senior. Belbin’s team role model: Development, validity and applications for team building. *Journal of Management Studies*, 44(1):96–118, 2007. DOI: 10.1111/j.1467-6486.2007.00666.x.
- [42] Edward de Bono. *Six Thinking Hats*. Little, Brown, 1985.
- [43] Lu Hong and Scott E. Page. Groups of diverse problem solvers can outperform groups of high-ability problem solvers. *Proceedings of the National Academy of Sciences*, 101(46):16385–16389, 2004. DOI: 10.1073/pnas.0403723101.
- [44] Abigail Thompson. Does diversity trump ability? An example of the misuse of mathematics in the social sciences. *Notices of the American Mathematical Society*, 61(9):1024–1030, 2014.
- [45] Daniel Kuehn. Diversity, ability, and democracy: A note on Thompson’s challenge to Hong and Page. *Critical Review*, 29(1):72–87, 2017. DOI: 10.1080/08913811.2017.1288455.
- [46] Miao Zhang, Junsik Kim, Siyuan Xiang, Jian Gao, and Cheng Cao. Dynamic role assignment for multi-agent debate. arXiv:2601.17152, 2026.
- [47] Zijun Liu, Yanzhe Zhang, Peng Li, Yang Liu, and Diyi Yang. A dynamic LLM-powered agent network for task-oriented agent collaboration. In *Proceedings of the 1st Conference on Language Modeling (COLM)*, 2024. arXiv:2310.02170 (DyLAN).
- [48] Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Dongsheng Li, and Deqing Yang. EvoAgent: Towards automatic multi-agent generation via evolutionary algorithms. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL)*, 2025. arXiv:2406.14228.
- [49] Han Zhou, Xingchen Wan, Ruoxi Sun, Hamid Palangi, Shariq Iqbal, Ivan Vulić, Anna Korhonen, and Sercan Ö. Arik. Multi-agent design: Optimizing agents with better prompts and topologies. In *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026. arXiv:2502.02533.
- [50] Yuntong Hu, Yuting Zhang, Matthew Trager, Yi Zhang, Shuo Yang, Wei Xia, and Stefano Soatto. EvoMAS: Evolutionary generation of multi-agent systems. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.06511.
- [51] Huaixiu Steven Zheng, Swaroop Mishra, Xinyun Chen, Heng-Tze Cheng, Ed H. Chi, Quoc V. Le, and Denny Zhou. Take a step back: Evoking reasoning via abstraction in large language models. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06117.
- [52] Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. arXiv:2305.04091.
- [53] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi. Least-to-most prompting enables complex reasoning in large language models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023. arXiv:2205.10625.

- [54] Yixuan Weng, Minjun Zhu, Fei Xia, Bin Li, Shizhu He, Shengping Liu, Bin Sun, Kang Liu, and Jun Zhao. Large language models are better reasoners with self-verification. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023. arXiv:2212.09561.
- [55] Michihiro Yasunaga, Xinyun Chen, Yujia Li, Panupong Pasupat, Jure Leskovec, Percy Liang, Ed H. Chi, and Denny Zhou. Large language models as analogical reasoners. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.01714.
- [56] Vighnesh Subramaniam, Yilun Du, Joshua B. Tenenbaum, Antonio Torralba, Shuang Li, and Igor Mordatch. Multiagent finetuning: Self improvement with diverse reasoning chains. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025. arXiv:2501.05707.
- [57] Ankur Samanta, Akshayaa Magesh, Runzhe Wu, Ayush Jain, Youliang Yu, Daniel Jiang, Boris Vidolov, Paul Sajda, Yonathan Efroni, and Kaveh Hassani. Self-improvement of language models by post-training on multi-agent debate. arXiv:2509.15172, 2025.
- [58] John Gkountouras, Josip Jukić, and Ivan Titov. Consensus as privileged context for label-free self-distillation. arXiv:2607.13643, 2026.
- [59] Wanjia Zhao, Mert Yuksekgonul, Shirley Wu, and James Zou. SiriuS: Self-improving multi-agent systems via bootstrapped reasoning. In *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025. arXiv:2502.04780.
- [60] Gaurav Srivastava, Zhenyu Bi, Meng Lu, and Xuan Wang. DEBATE, TRAIN, EVOLVE: Self-evolution of language model reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 32764–32810, 2025. DOI: 10.18653/v1/2025.emnlp-main.1666, arXiv:2505.15734.
- [61] Chanwoo Park, Seungju Han, Xingzhi Guo, Asuman Ozdaglar, Kaiqing Zhang, and Joo-Kyung Kim. MAPoRL: Multi-agent post-co-training for collaborative large language models with reinforcement learning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2502.18439.
- [62] Chenxi Liu, Yanshuo Chen, Ruibo Chen, Tianyi Xiong, Tong Zheng, and Heng Huang. Learning from self-debate: Preparing reasoning models for multi-agent debate. arXiv:2601.22297, 2026.
- [63] Martino M. L. Pulici, Cuong Xuan Chu, Evgeny Kharlamov, Zifeng Ding, Volker Tresp, and Yunpu Ma. MADA-RL: Multi-agent debate-aware reinforcement learning for parameter-efficient reasoning in compact models. arXiv:2607.18006, 2026.
- [64] Geoffrey E. Hinton and Steven J. Nowlan. How learning can guide evolution. *Complex Systems*, 1(3):495–502, 1987.
- [65] Max Jaderberg et al. Population based training of neural networks. arXiv:1711.09846, 2017.

- [66] Jonathan Cook, Chris Lu, Edward Hughes, Joel Z. Leibo, and Jakob Foerster. Artificial generational intelligence: Cultural accumulation in reinforcement learning. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024. DOI: 10.52202/079017-1907, arXiv:2406.00392.
- [67] Aron Vallinder and Edward Hughes. Cultural evolution of cooperation among LLM agents. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 2771–2773, 2025. Extended abstract. arXiv:2412.10270.
- [68] Takuya Akiba, Makoto Shing, Yujin Tang, Qi Sun, and David Ha. Evolutionary optimization of model merging recipes. *Nature Machine Intelligence*, 2025. arXiv:2403.13187.
- [69] Chengsong Huang et al. LoraHub: Efficient cross-task generalization via dynamic LoRA composition. In *Proceedings of the 1st Conference on Language Modeling (COLM)*, 2024. arXiv:2307.13269.
- [70] Shangbin Feng et al. Model swarms: Collaborative search to adapt LLM experts via swarm intelligence. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2410.11163.
- [71] Yiqun Zhang, Peng Ye, Xiaocui Yang, Shi Feng, Shufei Zhang, Lei Bai, Wanli Ouyang, and Shuyue Hu. Nature-inspired population-based evolution of large language models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 44187–44205, 2026. DOI: 10.18653/v1/2026.acl-long.2044, arXiv:2503.01155.
- [72] Dongxin Guo, Jikun Wu, and Siu Ming Yiu. EvoPref: Multi-objective evolutionary optimization discovers diverse LLM alignments beyond gradient descent. arXiv:2605.09777, 2026.
- [73] Roger Creus Castanyer, Geoffrey Bradway, Lorenz Wolf, Maxwell Lin, Augustine N. Mavor-Parker, and Matthew James Sargent. PopuLoRA: Co-evolving LLM populations for reasoning self-play. arXiv:2605.16727, 2026.
- [74] Xin Qiu, Yulu Gan, Conor F. Hayes, Qiyao Liang, Yinggan Xu, Roberto Dailey, Elliot Meyerson, Babak Hodjat, and Risto Miikkulainen. Evolution strategies at scale: LLM fine-tuning beyond reinforcement learning. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2509.24372.
- [75] Shangbin Feng, Zifeng Wang, Palash Goyal, Yike Wang, Weijia Shi, Huang Xia, Hamid Palangi, Luke Zettlemoyer, Yulia Tsvetkov, Chen-Yu Lee, and Tomas Pfister. Heterogeneous swarms: Jointly optimizing model roles and weights for multi-LLM systems. In *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025. arXiv:2502.04510.
- [76] Andrew Dai, Boris Meinardus, Ciaran Regan, Yingtao Tian, and Yujin Tang. Discovering novel LLM experts via task-capability coevolution. In *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026. arXiv:2604.14969.
- [77] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. arXiv:1504.04909, 2015.

- [78] So Kuroki, Taishi Nakamura, Takuya Akiba, and Yujin Tang. Agent skill acquisition for large language models via CycleQD. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.14735.
- [79] Mitchell Wortsman et al. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022. arXiv:2203.05482.
- [80] Gabriel Ilharco et al. Editing models with task arithmetic. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023. arXiv:2212.04089.
- [81] Prateek Yadav et al. TIES-Merging: Resolving interference when merging models. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. arXiv:2306.01708.
- [82] Le Yu et al. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2311.03099 (DARE).
- [83] Enneng Yang, Li Shen, Guibing Guo, Xingwei Wang, Xiaochun Cao, Jie Zhang, and Dacheng Tao. Model merging in LLMs, MLLMs, and beyond: Methods, theories, applications, and opportunities. *ACM Computing Surveys*, 2026. arXiv:2408.07666, DOI: 10.1145/3787849.
- [84] George Stoica, Pratik Ramesh, Boglarka Ecsedi, Leshem Choshen, and Judy Hoffman. Model merging with SVD to tie the Knots. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.19735 (KnOTS).
- [85] Zhan Zhuang, Xiequn Wang, Wei Li, Yulong Zhang, Qiushi Huang, Shuhao Chen, Xuehao Wang, Yanbin Wei, Yuhe Nie, Kede Ma, Yu Zhang, and Ying Wei. Come together, but not right now: A progressive strategy to boost low-rank adaptation. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2506.05713.
- [86] Alejandro R. Salamanca, Ahmet Üstün, Nicki Skafte Detlefsen, and Tim Dettmers. Seeded LoRA: Collaborative fine-tuning through seed initialization of adapters. In *ICML 2024 Workshop on Efficient Systems for Foundation Models (ES-FoMo II)*, 2024. OpenReview: U3UtvOYMiW.
- [87] Lloyd S. Shapley. A value for  $n$ -person games. In *Contributions to the Theory of Games II*. Princeton University Press, 1953.
- [88] Amirata Ghorbani and James Zou. Data Shapley: Equitable valuation of data for machine learning. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019. arXiv:1904.02868.
- [89] Benedek Rozemberczki et al. The Shapley value in machine learning. In *Proceedings of the 31st International Joint Conference on Artificial Intelligence (IJCAI), Survey Track*, 2022.
- [90] John F. Banzhaf, III. Weighted voting doesn't work: A mathematical analysis. *Rutgers Law Review*, 19:317–343, 1965.

- [91] Jiachen T. Wang and Ruoxi Jia. Data Banzhaf: A robust data valuation framework for machine learning. In *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2023. arXiv:2205.15466.
- [92] David H. Wolpert and Kagan Tumer. An introduction to collective intelligence. arXiv:cs/9908014, 1999.
- [93] Adrian K. Agogino and Kagan Tumer. Efficient evaluation functions for evolving coordination. *Evolutionary Computation*, 16(2):257–288, 2008.
- [94] Jakob Foerster et al. Counterfactual multi-agent policy gradients. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, 2018. arXiv:1705.08926 (COMA).
- [95] Catherine Rodriquez and James Ghawaly, Jr. Co-evolved spiking neural network ensembles via marginal contribution fitness. arXiv:2606.13985, 2026. ACM International Conference on Neuromorphic Systems (ICONS) 2026.
- [96] Mitchell A. Potter and Kenneth A. De Jong. A cooperative coevolutionary approach to function optimization. In *Parallel Problem Solving from Nature (PPSN III)*. Springer, 1994.
- [97] R. Paul Wiegand, William C. Liles, and Kenneth A. De Jong. An empirical analysis of collaboration methods in cooperative coevolutionary algorithms. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, pages 1235–1242, 2001.
- [98] Liviu Panait, Sean Luke, and R. Paul Wiegand. Biasing coevolutionary search for optimal multiagent behaviors. *IEEE Transactions on Evolutionary Computation*, 10(6):629–645, 2006.
- [99] Yanming Li et al. Who deserves the reward? SHARP: Shapley credit-based optimization for multi-agent system. arXiv:2602.08335, 2026.
- [100] Yanjun Chen, Yirong Sun, Hanlin Wang, Jinghan Wang, Xinming Zhang, Xiaoyu Shen, Wenjie Li, and Wei Zhang. The trace is the state: Exact credit assignment for LLM agent teams. arXiv:2603.06859, 2026.
- [101] Mingyu Lu, Yushan Huang, Chris Lin, and Su-In Lee. Agents that matter: Optimizing multi-agent LLMs via removal-based attribution. arXiv:2605.27621v1, 2026. Later versions are titled “Agents that Matter: How Subtle Choices Shape Agent Attribution in Multi-Agent Systems”.
- [102] Yichi Zhang, Kevin Lu, Yuang Zhang, Jie Gao, Lirong Xia, and Fang-Yi Yu. Mixture of complementary agents for robust LLM ensemble. arXiv:2605.24048, 2026.
- [103] Rich Caruana, Alexandru Niculescu-Mizil, Geoff Crew, and Alex Ksikes. Ensemble selection from libraries of models. In *Proceedings of the 21st International Conference on Machine Learning (ICML)*, 2004.
- [104] Anders Krogh and Jesper Vedelsby. Neural network ensembles, cross validation, and active learning. In *Advances in Neural Information Processing Systems 7 (NeurIPS)*, 1995.
- [105] Danny Wood, Tingting Mu, Andrew Webb, Henry Reeve, Mikel Luján, and Gavin Brown. A unified theory of diversity in ensemble learning. *Journal of Machine Learning Research*, 24, 2023. arXiv:2301.03962.

- [106] Donghwan Kim. Are diversity metrics measuring diversity? A capability-controlled audit of majority-vote gain in LLM ensembles. arXiv:2607.20768, 2026.
- [107] Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations. arXiv:2411.00640, 2024.
- [108] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker’s guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018.
- [109] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2), 1979.
- [110] Sida Wang. Measuring all the noises of LLM evals. arXiv:2512.21326, 2025.
- [111] Yaochu Jin and Jürgen Branke. Evolutionary optimization in uncertain environments—a survey. *IEEE Transactions on Evolutionary Computation*, 9(3):303–317, 2005.
- [112] Pratyusha Rakshit, Amit Konar, and Swagatam Das. Noisy evolutionary optimization algorithms – a comprehensive survey. *Swarm and Evolutionary Computation*, 33:18–45, 2017.
- [113] Yang Xu, Jiefu Zhang, Haixiang Sun, Zihan Zhou, Tianyu Cao, and Vaneet Aggarwal. Towards reliable LLM evaluation: Correcting the winner’s curse in adaptive benchmarking. arXiv:2605.05973, 2026.
- [114] Felipe Maia Polo et al. tinyBenchmarks: Evaluating LLMs with fewer examples. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.14992.
- [115] Liam Cooper, Shinnung Jeong, Hyeran Jeon, Jeffrey Young, and Hyesoon Kim. Accelerating the mitigation of LLM inference nondeterminism across GPU architectures. arXiv:2609.25624, 2026.
- [116] Nada Zine, Tristan Coignon, Vincenzo Stoico, Clément Quinton, Ivano Malavolta, Romain Rouvoy, and Patricia Lago. Attention to detail: Evaluating energy, performance, and accuracy trade-offs across vLLM configurations. arXiv:2607.09172, 2026.
- [117] Hynek Kydlíček. Math-Verify: Math verification library. <https://github.com/huggingface/Math-Verify>, 2025. Version 0.8.0.
- [118] Qian Zhang, Yan Zheng, Jinyi Liu, Hebin Liang, and Lanjun Wang. Key decision-makers in multi-agent debates: Who holds the power? arXiv:2511.11040, 2025.
- [119] Nathan Halko, Per-Gunnar Martinsson, and Joel A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2), 2011. arXiv:0909.4061.
- [120] Woosuk Kwon et al. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023. arXiv:2309.06180 (vLLM).

- [121] Yubo Wang et al. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*, 2024. arXiv:2406.01574.
- [122] M-A-P Team, Xinrun Du, et al. SuperGPQA: Scaling LLM evaluation across 285 graduate disciplines. arXiv:2502.14739, 2025.
- [123] Dan Hendrycks et al. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems 34 (NeurIPS), Datasets and Benchmarks Track*, 2021. arXiv:2103.03874.
- [124] Karl Cobbe et al. Training verifiers to solve math word problems. arXiv:2110.14168, 2021.
- [125] Hunter Lightman et al. Let’s verify step by step. arXiv:2305.20050 (ICLR 2024), 2023.

# 付 録 A 実験で用いたプロンプト

本付録では、実験で用いたプロンプトの全文を示す。英語の文は実装のとおりであり、ペルソナ文は `persona_pool.json` に、議論のプロンプトは `prompts.py` に定義されている。[ ] は実際の内容が入る箇所、`\n` は改行を表す。

## A.1 システムプロンプトと回答書式

システムプロンプトは、ペルソナ文と回答書式の指示を空行でつないだものである（plain ではペルソナ文を置かない）。

### システムプロンプト

[ペルソナ文] `\n\n` [回答書式の指示]

回答書式の指示は、問題の種類ごとに次のとおりである。

### 多肢選択

Think step by step. Then give your final answer on the last line in exactly this format:`\nANSWER: <letter>`

### 数学

Think step by step. Then give your final answer on the last line in exactly this format:`\nANSWER: <final simplified answer>`

多肢選択の問題文は、問題の本文、空行、「A. [選択肢]」「B. [選択肢]」…の各行からなる。

## A.2 ペルソナ文

世代0の選定の候補（3.2.1 項）のペルソナ文を次に示す。plain はペルソナ文を持たない。

Plant

You are a creative idea generator. Before settling on an answer, consider at least two different interpretations of the question or two different ways to solve it, then choose the best-supported one.

### **Monitor Evaluator**

You are a critical evaluator. Test each answer option or intermediate claim against the conditions of the problem one by one, and eliminate those that fail before committing to an answer.

### **Specialist**

You are a domain specialist. First state the key principle, definition, or formula that governs this problem, then apply it step by step.

### **Shaper**

You are a decisive challenger. Commit early to the most likely answer, then deliberately try to refute it by attacking its weakest assumption, and change it only if the refutation succeeds.

### **Implementer**

You are a disciplined implementer. Apply the most standard, direct method systematically, keep every step concrete, and sanity-check the result with a quick estimate.

### **Completer Finisher**

You are a meticulous finisher. Solve the problem, then verify your answer backwards against every condition (substitute it back or re-derive it another way) and fix any error you find.

### **Co-ordinator**

You are a coordinator. Break the problem into a short plan of sub-questions, answer them in order, and combine the partial results into the final answer.

### **Teamworker**

You are a cooperative team player. Explain your reasoning plainly so that others can follow and check it, and resolve the most likely misunderstanding of the question before answering.

### **Resource Investigator**

You are a resourceful investigator. Recall a similar problem, example, or well-established fact you are confident about, and adapt its solution to this problem.

7月のチーム（4.4.3項）のペルソナ文は、予備研究で用いた次の日本語の文である。

**critic** あなたは厳密な検証を重視する批判的思考家。反証・例外・境界条件に敏感。

**pragmatist** あなたは応用志向の実務家。意思決定に役立つ実装可能性とコストを重視。

**explorer** あなたは創発を促す発想家。仮説生成と多角的比喻で発想を広げる。

Belbinの9役割とde Bonoの6つの思考モード [42] の対応を表 A.1 に示す。9役割は6つのモードのすべてに対応する役割を含む。

表 A.1: Six Thinking Hats の思考モードと候補の役割の対応

| 帽子 | 思考モード | 対応する役割                                |
|----|-------|---------------------------------------|
| 白  | 事実・情報 | Specialist, Resource Investigator     |
| 赤  | 直観    | Shaper                                |
| 黒  | 慎重・批判 | Monitor Evaluator, Completer Finisher |
| 黄  | 利点・肯定 | Teamworker                            |
| 緑  | 創造    | Plant                                 |
| 青  | 進行の管理 | Co-ordinator                          |

## A.3 round1のプロンプト

本研究のプロトコル（3.3節）では、round1の入力は次の4つの発話からなる。

1. system: [システムプロンプト]
2. user: [問題文]
3. assistant: [自分のround0の回答]
4. user: 次の文.

### round1のuserの発話

```
Here are solutions to the same problem from other agents:\n\n---\nAgent 1 ---\n [他者の回答] \n\n--- Agent 2 ---\n [他者の回答] \n\n [更新の指示]
```

切り詰め版では、自分と他者の回答が3,000文字を超える場合に末尾3,000文字だけを残し、先頭に「[... earlier part of the solution omitted ...]」と改行を付ける。2体の連合では、他者の回答は Agent 1 の1つだけになる。

更新の指示は次のとおりである。

### 中立版（採用）

Compare their reasoning with your own solution above. Re-check the key steps and point out any errors, whether in your solution or in theirs. Then give your updated step-by-step solution. You may keep or change your answer.

### 条件付き版

Compare their reasoning with your own solution above. Re-check the key steps of your own solution. Change your answer only if you find a concrete error in your own reasoning, and state that error explicitly; otherwise keep your answer even if the other agents disagree. Then give your final step-by-step solution.

7月の手続き（7月のチームの評価と、4.6.2項の参照に用いた）では、round1の入力は system と user の2つの発話からなり、自分の round0 の回答は提示しない。user の発話は次のとおりである。

### 7月の手続きの user の発話

Question:\n [問題文] \n\nHere are solutions from other agents:\n\n---  
Agent 1 ---\n [他者の回答] \n\n--- Agent 2 ---\n [他者の回答]  
\n\nCarefully examine the other agents' reasoning and compare it with your own. First, briefly re-derive the key steps of your own solution. Change your answer ONLY if you can identify a concrete, specific error in your own reasoning, and state that error explicitly. If you cannot find a specific error in your own reasoning, keep your original answer even if the other agents disagree. Then provide your final step-by-step solution.